JOBLINK

INTELLIGENT RECRUITMENT PLATFORM

&

AI-POWERED APPLICANT TRACKING SYSTEM

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT

OF REQUIREMENT

FOR THE AWARD OF THE DEGREE

MASTER OF COMPUTER APPLICATIONS (MCA)

OF

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

BY

M AKASH

RegNo : 24PMC132

MARIAN COLLEGE
KUTTIKKANAM
AUTONOMOUS
MAKING COMPLETE

Marian College Kuttikanam Autonomous

Peermade, Kerala – 685 531

2026

JOBLINK

INTELLIGENT RECRUITMENTPLATFORM

&

AI-POWERED APPLICANT TRACKING SYSTEM

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT

OF REQUIREMENT

FOR THE AWARD OF THE DEGREE

MASTER OF COMPUTER APPLICATIONS

(MCA)

OF

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

BY

M AKASH

Reg No : 24PMC132

MARIAN COLLEGE
KUTTIKKANAM
AUTONOMOUS
MAKING COMPLETE

Marian College Kuttikanam Autonomous

Peermade, Kerala – 685 531

2026

A Project Report on

JOBLINK

INTELLIGENT RECRUITMENT PLATFORM

&

AI-POWERED APPLICANT TRACKING SYSTEM

A PROJECT REPORT SUBMITTED IN PARTIAL FULFILMENT OF REQUIREMENT
FOR THE AWARD OF THE DEGREE

MASTER OF COMPUTER APPLICATIONS (MCA)

OF

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

BY

M AKASH

24PMC132

Under the guidance of

M Reny Jose

Assistant Professor

PG Department of Computer Applications

Marian College Kuttikkanam Autonomous

MARIAN COLLEGE
KUTTIKKANAM
AUTONOMOUS
MAKING COMPLETE

Marian College Kuttikanam Autonomous

Peermade, Kerala – 685 531

2026

PG DEPARTMENT OF COMPUTER APPLICATIONS

Marian College Kuttikkanam Autonomous

MAHATMA GANDHI UNIVERSITY, KOTTAYAM

KUTTIKKANAM – 685 531, KERALA.

CERTIFICATE

This is to certify that the project work entitled

JOBLINK

is a bonafide record of work done by

M AKASH

Reg No : 24PMC132

In partial fulfillment of the requirements for the award of Degree of

MASTER OF COMPUTER APPLICATIONS [MCA]

During the academic year 2025-2026

Reny Jose
Assistant Professor
PG Department of Computer Applications
Marian College Kuttikkanam Autonomous

Mr Win Mathew John
Head of the Department
PG Department of Computer Applications
Marian College Kuttikkanam Autonomous

External Examiner

ACKNOWLEDGEMENT

I express my sincere gratitude to God Almighty for granting me the strength, wisdom, and determination to successfully complete this project. This journey has been one of continuous learning and growth, and I am grateful for the opportunity to work on a meaningful real-world solution.

I extend my heartfelt thanks to **Mr. Win Mathew John**, Head of the Department of Computer Applications, for his valuable support, encouragement, and vision in fostering an environment conducive to innovation and excellence. His leadership has been instrumental in guiding students toward impactful project work.

I am deeply grateful to **Reny Jose**, Assistant Professor, Department of Computer Applications, for her continuous guidance, patience, and constructive feedback throughout the development of this project. Hertechnical insights, practical suggestions, and unwavering support helped me navigate complex challenges and refine the system architecture. Her mentorship extended beyond technical aspects, teaching me the importance of systematic problem-solving and professional software development practices.

I also express my sincere thanks to all faculty members of the PG Department of Computer Applications for their constant support, academic guidance, and encouragement. Their commitment to quality education has provided me with a strong foundation in computer science and software engineering principles.

My heartfelt appreciation goes to my family for their endless patience, understanding, and moral support during the completion of this work. Their belief in my abilities motivated me to persevere through challenges. I am equally grateful to my friends and peers for their collaboration, feedback, and encouragement throughout this academic journey.

Finally, I acknowledge the open-source community and the developers of the technologies used in this project, whose contributions have made modern software development accessible and empowering.

M Akash

ABSTRACT

In the modern digital employment ecosystem, recruitment has become increasingly data-driven yet remains inefficient due to reliance on manual screening and keyword-based Applicant Tracking Systems (ATS). Organizations often receive hundreds or thousands of applications for a single job posting, making manual evaluation time-consuming and prone to inconsistency. Most existing recruitment platforms use keyword-matching techniques that fail to capture contextual meaning, resulting in inaccurate shortlisting and the rejection of potentially qualified candidates. Furthermore, the lack of transparency in evaluation processes creates uncertainty for applicants and reduces trust in digital hiring systems.

JOBLINK is an AI-powered recruitment and resume screening platform designed to modernize and optimize the hiring process through intelligent semantic analysis and structured workflow management. The system integrates Machine Learning models to compare resumes and job descriptions based on contextual similarity rather than keyword frequency. A hybrid scoring mechanism combines semantic similarity, structured skill overlap using standardized occupational datasets, and domain relevance weighting to produce transparent and balanced match scores.

The platform supports role-based access for Job Seekers, Recruiters, and Administrators. It provides resume upload and parsing, automated AI-based scoring, structured ATS pipeline management, notification handling, and real-time recruiter–candidate communication. Built using a microservices architecture with React.js, Node.js, Python FastAPI, and PostgreSQL, JOBLINK ensures scalability, modularity, and security. By integrating Artificial Intelligence with full-stack development technologies, the system enhances recruitment efficiency, improves matching accuracy, and establishes a transparent and intelligent hiring ecosystem.

TABLE OF CONTENTS (Full Stack)

1. INTRODUCTION 1
2. FUNCTIONAL REQUIREMENTS 4
3. NON-FUNCTIONAL REQUIREMENTS 9
4. UML DIAGRAMS 13
5. TEST CASES 26
6. INPUT DESGIN AND OUTPUT DESIGN 33
7. SYSTEM IMPLEMENTATION 40
8. AI INTEGRATION(MODULE DESCRIPTION /APIs USED) 47
9. FUTURE ENHANCEMENTS 53
10. CONCLUSION 57
11. ANNEXURE 59

TABLE INDEX

Table NO	Table Title	Page No
5.1	User Management Test Cases	27
5.2	Job Seeker Test Cases	28
5.3	Recruiter Test Cases	29
5.4	AI Engine Test Cases	30
5.5	Chat System Test Cases	31
5.6	Admin Test Cases	32

FIGURE INDEX

Figure No	Figure Title	Page No
4.2	Use Case Diagram	20
4.3.1	Job Seeker Activity Diagram	21
4.3.2	Recruiter Activity Diagram	22
4.3.3	Admin Activity Diagram	23
4.4	Class Diagram	24
4.5	Sequence Diagram	25

1. INTRODUCTION

INTRODUCTION

1.1 Background of the Study

Recruitment is a critical organizational function that determines the quality of human resources within a company. With the growth of digital job portals, organizations receive hundreds or even thousands of applications for a single job posting. Manual screening of resumes has become inefficient and time-consuming.

Although many companies use Applicant Tracking Systems, most systems rely on basic keyword matching techniques. These systems fail to understand semantic meaning, contextual relevance, and transferable skills. As a result, qualified candidates are often filtered out due to variations in terminology or resume structure.

The integration of Artificial Intelligence and Natural Language Processing provides an opportunity to enhance recruitment systems. Semantic embedding models such as Sentence-BERT allow machines to understand contextual similarity rather than simple word overlap. JobLink leverages these technologies to create an intelligent recruitment ecosystem.

1.2 Proposed System

Traditional recruitment systems face multiple challenges. Manual screening consumes significant time and resources, particularly for large organizations. Keyword-based filtering methods lack contextual intelligence, leading to inaccurate candidate evaluation. Recruiters often struggle to maintain fairness and consistency when reviewing large numbers of applications. Additionally, candidates receive limited feedback regarding their application status, resulting in reduced transparency and engagement.

There is a need for an intelligent system capable of understanding resume content semantically, validating structured skills, and managing recruitment workflows efficiently

1.3 Objectives of the Project

JOBLINK is proposed as an intelligent recruitment platform that enhances resume screening through semantic similarity modeling and structured evaluation techniques. The system

integrates Artificial Intelligence to compare resumes and job descriptions at a contextual level rather than relying solely on keyword presence.

The platform provides a complete recruitment workflow, beginning with candidate registration and resume upload, followed by automated AI scoring, application submission, recruiter review, and status updates. Recruiters can post job openings with detailed skill requirements, and the system evaluates each applicant using a hybrid scoring algorithm. Real-time communication features allow interaction between recruiters and candidates within the platform.

By combining AI-powered analysis with a structured Applicant Tracking System, JOBLINK reduces manual effort, improves shortlisting accuracy, and enhances transparency in recruitment decisions.

1.4 Scope of the Project

JOBLINK supports three user roles: Job Seekers, Recruiters, and Administrators. The system allows job seekers to create accounts, upload resumes, browse job postings, and apply for positions. Recruiters can post job listings, evaluate applicants using AI-generated match scores, and manage applications through structured stages. Administrators monitor platform operations and manage system integrity.

The project focuses on intelligent resume screening and recruitment workflow management. Advanced features such as video interviews and third-party HR integrations are considered future enhancements.

2. FUNCTIONAL REQUIREMENTS

FUNCTIONAL REQUIREMENTS

The JOBLINK system shall provide the following core functionalities to support intelligent recruitment, secure user management, AI-powered resume evaluation, real-time communication, and administrative oversight.

2.1 User Management

The system shall provide secure user registration and authentication using JSON Web Tokens (JWT). During registration, the system shall validate email format and ensure uniqueness before account creation. Passwords shall be securely hashed using bcrypt with a salt factor of 10 prior to storage in the database.

Role-Based Access Control (RBAC) shall be implemented to restrict access based on user roles: **Seeker, Recruiter, and Admin**. Access control shall be enforced at two levels: frontend route wrapper components and backend middleware verification for protected endpoints.

Upon first successful login, users shall be redirected to an onboarding flow where they must complete essential profile information before accessing the main dashboard. This ensures that all users maintain a minimum profile completeness level.

2.2 Job Seeker Functions

The system shall allow job seekers to create and manage comprehensive profiles including personal details, professional headline, location, about section, skills, education history, resume (PDF format), and profile picture.

The job browsing module shall allow filtering by employment type, including Full-time, Part-time, Contract, and Internship positions. A “Clear All” option shall reset active filters instantly.

Job seekers shall be able to apply to a job using a one-click submission process with PDF resume upload. The system shall prevent duplicate applications to the same job posting.

Upon application submission, the system shall automatically invoke the Machine Learning (ML) Engine to compute a hybrid ATS score. The computed score and skill breakdown shall be stored in the Applications table and displayed within the **My Applications dashboard**, along with real-time status updates.

Additionally, job seekers shall have access to:

- A **Resume Analyzer** tool for ad-hoc ATS scoring against any pasted job description.
- A **Smart Match** feature that generates the top five recommended career roles based on the uploaded resume.

2.3 Recruiter Functions

Recruiters shall be able to create detailed job postings containing more than twelve configurable fields, including tag-based inputs for requirements, responsibilities, and benefits.

For each posted job, recruiters shall be able to view all applicants along with:

- AI match score
- Match level label
- Matched skills
- Missing skills
- Resume download link
- Current pipeline stage

Recruiters shall update applicant pipeline stages (Under Review, Interviewed, Rejected, Accepted). Each stage transition shall automatically update the database and trigger an in-app notification to the respective seeker.

A **Message** button shall be available on each applicant card, enabling recruiters to initiate real-time communication through a Socket.IO-powered chat system directly from the applications interface.

2.3 AI Integration

The system shall extract text from uploaded PDF resumes using the **pdf-parse** library on the Node.js backend. The extracted resume text and corresponding job description shall be forwarded to a locally hosted Python-based ML Engine via HTTP POST request.

The ML Engine shall compute a hybrid ATS score composed of:

- 60% SBERT semantic similarity
- 30% O*NET skill overlap
- 10% domain-specific boost across nine technology domains

The engine shall return:

- Overall score
- Match level label
- Semantic score
- Keyword score
- Matched skills
- Missing skills
- Personalized improvement recommendation

The suggest-roles endpoint shall return the top five recommended job roles with individual component scores and confidence levels.

If the ML Engine becomes unavailable, the backend shall automatically activate a keyword-based fallback scoring heuristic to ensure application submission does not fail.

2.4 Real-Time Communication

The system shall implement a Socket.IO-based chat server running on port 3001. Deterministic room IDs shall be generated by sorting and combining the two participants' integer user IDs.

All messages shall be stored in a PostgreSQL **messages** table with seen and deleted flags for read receipts and soft deletion.

The chat server shall handle:

- Room join events with message history loading
- Sending and broadcasting messages
- Marking messages as read with read receipt emission
- Soft-deleting messages

Each message sent shall trigger creation of a corresponding Notification record in the database. A deduplication mechanism shall suppress duplicate notifications within a two-minute interval.

2.5 Notification System

The system shall maintain a **Notifications** table containing:

- Type
- Title
- Message
- RelatedLink
- Read status

Notifications shall be generated by both the application status update workflow and the chat server. Each notification shall contain a deep link redirecting the user to the relevant page or conversation.

A notification bell displaying the unread count shall be present in both recruiter and seeker navigation layouts.

2.6 Administrator Functions

The system shall provide an Admin Dashboard presenting platform-wide aggregate statistics.

Administrators shall be able to:

- View all registered users with role-based filtering
- View all job postings across all recruiters

All admin routes shall be protected, ensuring only users with the Admin role can access moderation and oversight features.

3. NON-FUNCTIONAL REQUIREMENTS

NON-FUNCTIONAL REQUIREMENTS

3.1 Performance

The ML Engine shall pre-compute all job role embeddings at startup to maintain inference latency under 100 milliseconds per request. The SBERT model shall be loaded once during initialization and reused across subsequent requests to eliminate repeated overhead.

Standard API response times shall remain under 500 milliseconds, while AI-powered analysis shall complete within 2 seconds.

Chat history loading shall be limited to the most recent 100 messages per room to reduce data transfer. The React frontend shall utilize Vite for efficient development and optimized production bundling.

3.2 Security

All passwords shall be hashed using bcryptjs with 10 salt rounds before storage. Plain-text passwords shall never be persisted.

JWT tokens shall be required in the Authorization header using the Bearer scheme for all protected endpoints. Role-based access control shall be enforced at both frontend and backend layers.

File uploads shall be validated using Multer middleware, restricting uploads to PDF format and enforcing file size limits.

All sensitive configuration values (database credentials, JWT secrets, API keys) shall be stored exclusively in `.env` files and excluded from version control.

3.3 Reliability

The system shall implement graceful fallback mechanisms at every integration boundary. If the Python ML Engine is unreachable, the backend shall activate a keyword-based scoring fallback to prevent application submission failure.

If port 3001 is occupied, the chat server shall log a warning and skip initialization without crashing the primary API process.

All asynchronous database operations shall be wrapped in try-catch blocks with appropriate HTTP responses and logging to prevent unhandled exceptions.

3.4 Scalability

The architecture shall follow a microservices-inspired design where the ML Engine, Chat Server, and REST API operate as independent services. Each component may be scaled independently.

PostgreSQL shall use indexed primary and foreign keys to ensure efficient query execution as data volume increases.

Stateless JWT authentication shall enable horizontal scaling without session sharing requirements.

3.5 Usability

Role-specific layouts (SeekerLayout, RecruiterLayout, AdminLayout) shall provide intuitive navigation tailored to each user type.

Real-time feedback shall be provided using toast notifications for validation errors and success confirmations.

The chat interface shall include read receipts, soft-delete indicators, and consistent avatar representation. The system shall maintain responsive design compatibility across desktop and laptop devices.

3.6 Maintainability

The backend shall follow a modular directory structure separating routes, controllers, models, middleware, and utilities.

Frontend components shall be organized into role-based directories. The ML Engine shall operate as an independent Python package with its own environment and dataset directory.

All environment-specific configurations shall be externalized to `.env` files. API endpoints shall be logically separated into six route modules to ensure clarity and extensibility.

4. UML DIAGRAMS

UML DIAGRAMS

4.1 Use Cases

Use Case 1: Job Seeker Registration and Onboarding

• **Use Case ID:** UC-001

• **Actors:** Visitor (Unregistered User)

• **Goal:** Create a new account and gain access to the JOBLINK platform

• **Preconditions:**

– User has a valid email address

– User has internet connectivity

• **Postconditions:**

– User account is created in PostgreSQL

– Password is securely hashed

– JWT authentication token is generated

– User is redirected to Onboarding page

• **Main Flow:**

Navigate to Registration page → Enter Name, Email, Password → Select Role (Seeker or Recruiter) → (If Recruiter, enter Company Name) → Click Register → System validates email uniqueness → System hashes password using bcrypt → System creates user record → System generates JWT token → System redirects user to Onboarding page → User completes Headline, Location, Skills → System redirects to Dashboard.

• **Alternative Flow:**

– Email already exists → Error message displayed: "User with this email already exists."

– Password validation fails → Error message displayed.

– Required fields missing → Form validation error displayed.

Use Case 2: Job Seeker Browses and Filters Jobs

• **Use Case ID:** UC-002

• **Actors:** Job Seeker

• **Goal:** View and filter available job listings

• **Preconditions:**

– User is logged in as Seeker

• **Postconditions:**

– Filtered job listings are displayed

• **Main Flow:**

Navigate to Jobs page → System displays all active jobs → User selects filter (Full-time / Part-time / Contract / Internship) → System filters jobs client-side → Filtered jobs displayed → User may click "Clear All" to reset filters.

• **Alternative Flow:**

– No jobs match filter → System displays "No jobs found" message.

Use Case 3: Job Application Submission with AI Scoring

• **Use Case ID:** UC-003

• **Actors:** Job Seeker

• **Goal:** Apply for a job and receive AI match score

• **Preconditions:**

– User is logged in

– Job exists and is active

• **Postconditions:**

– Application record created

– AI score stored in Applications table

– Confirmation message displayed

• **Main Flow:**

Open Job Details page → Click Apply → Upload Resume (PDF) → System validates file type using Multer → System creates application record (status = Applied) → System extracts resume text using pdf-parse → Backend sends resume text and job description to ML Engine → ML Engine computes Hybrid ATS Score → System stores aiScore and aiFeedback → Confirmation with match score displayed.

• **Alternative Flow:**

– Resume not in PDF format → Error message displayed.

– Duplicate application → Error message displayed: "You have already applied for this job."
– ML Engine unavailable → System activates fallback keyword scoring.

Use Case 4: AI Resume Analyzer (Ad-Hoc Scoring)

• **Use Case ID:** UC-004
• **Actors:** Job Seeker
• **Goal:** Analyze resume against any job description without applying
• **Preconditions:**
– User is logged in
• **Postconditions:**
– ATS report displayed

• **Main Flow:**
Navigate to Resume Analyzer → Upload Resume (PDF) → Paste Job Description → Click Analyze → System extracts resume text → Backend calls ML Engine → ML Engine computes hybrid score → System displays overall score, semantic score, keyword score, matched skills, missing skills, and improvement suggestions.

• **Alternative Flow:**
– Invalid file format → Error displayed.
– ML Engine unavailable → Fallback scoring applied.

Use Case 5: Smart Career Match

• **Use Case ID:** UC-005
• **Actors:** Job Seeker
• **Goal:** Receive top five recommended career roles based on resume
• **Preconditions:**
– User is logged in
• **Postconditions:**
– Top five recommended roles displayed

• **Main Flow:**
Navigate to Smart Match page → Upload Resume (PDF) → Backend sends resume text to ML Engine suggest-roles endpoint → ML Engine performs classification → Returns top five

job roles → System displays role name, fit score, semantic score, skill score, domain boost, and confidence level.

• **Alternative Flow:**
– Resume upload fails → Error displayed.
– ML Engine unavailable → Error message shown.

Use Case 6: Recruiter Posts a Job

• **Use Case ID:** UC-006
• **Actors:** Recruiter
• **Goal:** Create a new job posting
• **Preconditions:**
– Recruiter is logged in
• **Postconditions:**
– Job stored in Jobs table
– Job visible to seekers

• **Main Flow:**
Navigate to Post Job page → Enter job details (Title, Description, Location, Salary, Requirements, Responsibilities, Benefits, Type, etc.) → Click Post Job → System validates fields → System saves job in PostgreSQL → Job becomes visible in Browse Jobs.

• **Alternative Flow:**
– Required fields missing → Validation error displayed.

Use Case 7: Recruiter Manages Applicant Pipeline

• **Use Case ID:** UC-007
• **Actors:** Recruiter
• **Goal:** Review applicants and update pipeline stage
• **Preconditions:**
– Recruiter has posted at least one job
• **Postconditions:**
– Application status updated
– Notification sent to seeker

• **Main Flow:**

Navigate to My Jobs → Open Job Applications → View applicant AI score and skills → Select new pipeline stage (Under Review / Interviewed / Rejected / Accepted) → System updates Applications table → System creates notification with deep link → Recruiter may click “Message” to open chat.

• **Alternative Flow:**

– Database update fails → Error message displayed.

Use Case 8: Real-Time Messaging

• **Use Case ID:** UC-008

• **Actors:** Recruiter, Job Seeker

• **Goal:** Communicate in real time

• **Preconditions:**

– Both users are logged in

• **Postconditions:**

– Message stored in database

– Notification created

• **Main Flow:**

Click Message → System generates room ID → Client emits join_room event → Server loads last 100 messages → User types message → send_message event emitted → Server stores message → Message broadcasted → Notification created → Recipient reads message → mark_read event updates seen flag.

• **Alternative Flow:**

– Chat server unavailable → Chat not initialized, system continues running.

Use Case 9: View Notifications

• **Use Case ID:** UC-009

• **Actors:** Job Seeker, Recruiter

• **Goal:** View and act on notifications

• **Preconditions:**

– User is logged in

• **Postconditions:**

– Notification marked as read

• **Main Flow:**

Click Notification Bell → System displays notifications list → Click notification → System redirects to relatedLink → PATCH request marks notification as read → Unread count updated.

• **Alternative Flow:**

– No notifications → Empty state message displayed.

Use Case 10: Admin Platform Oversight

• **Use Case ID:** UC-010

• **Actors:** Admin

• **Goal:** Monitor users and job postings

• **Preconditions:**

– Admin is logged in

• **Postconditions:**

– Admin views platform data

• **Main Flow:**

Login → Redirect to Admin Dashboard → View aggregate statistics → Navigate to Users page → Filter users by role → Navigate to Jobs page → Review job postings.

• **Alternative Flow:**

– Unauthorized access attempt → Redirect to home page.

4.2 Use Case Diagram

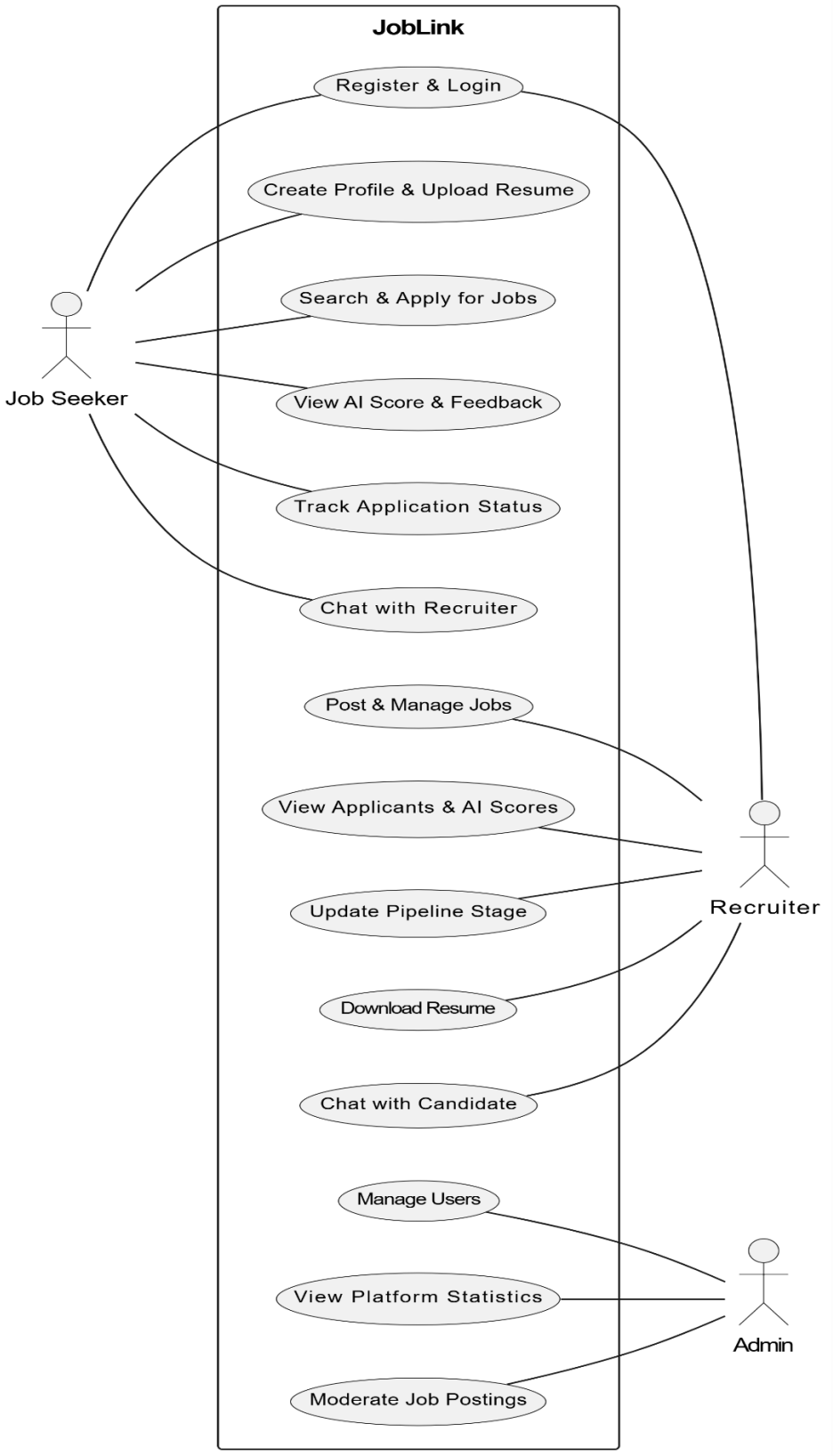


Fig 4.2 Use case Diagram

4.3 Activity Diagram

4.3.1 Jobseeker Activities:

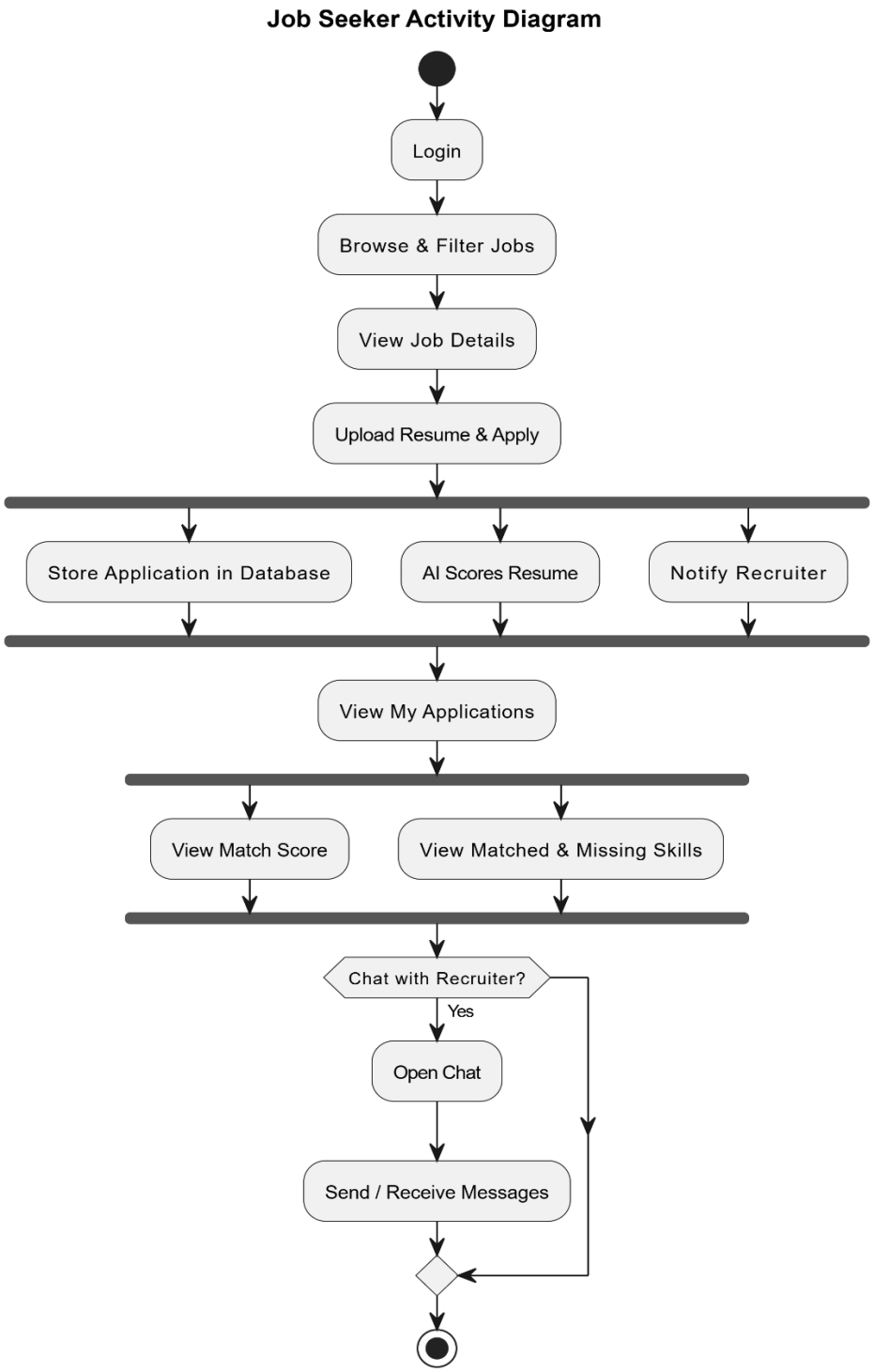


Fig 4.3.1 Jobseeker Activity Diagram

4.3.2 Recruiter Activities:

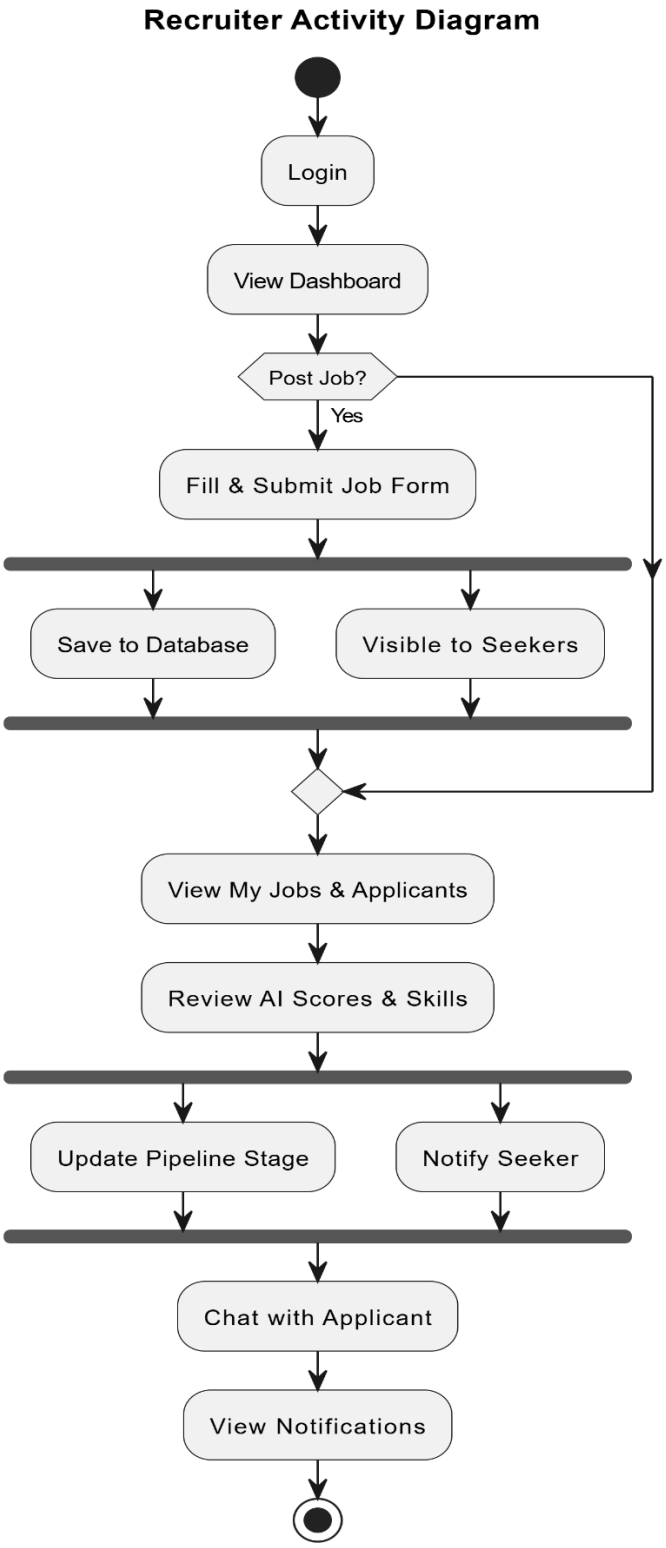


Fig 4.3.2 Recruiter Activity

4.3.3 Admin Activities*:*

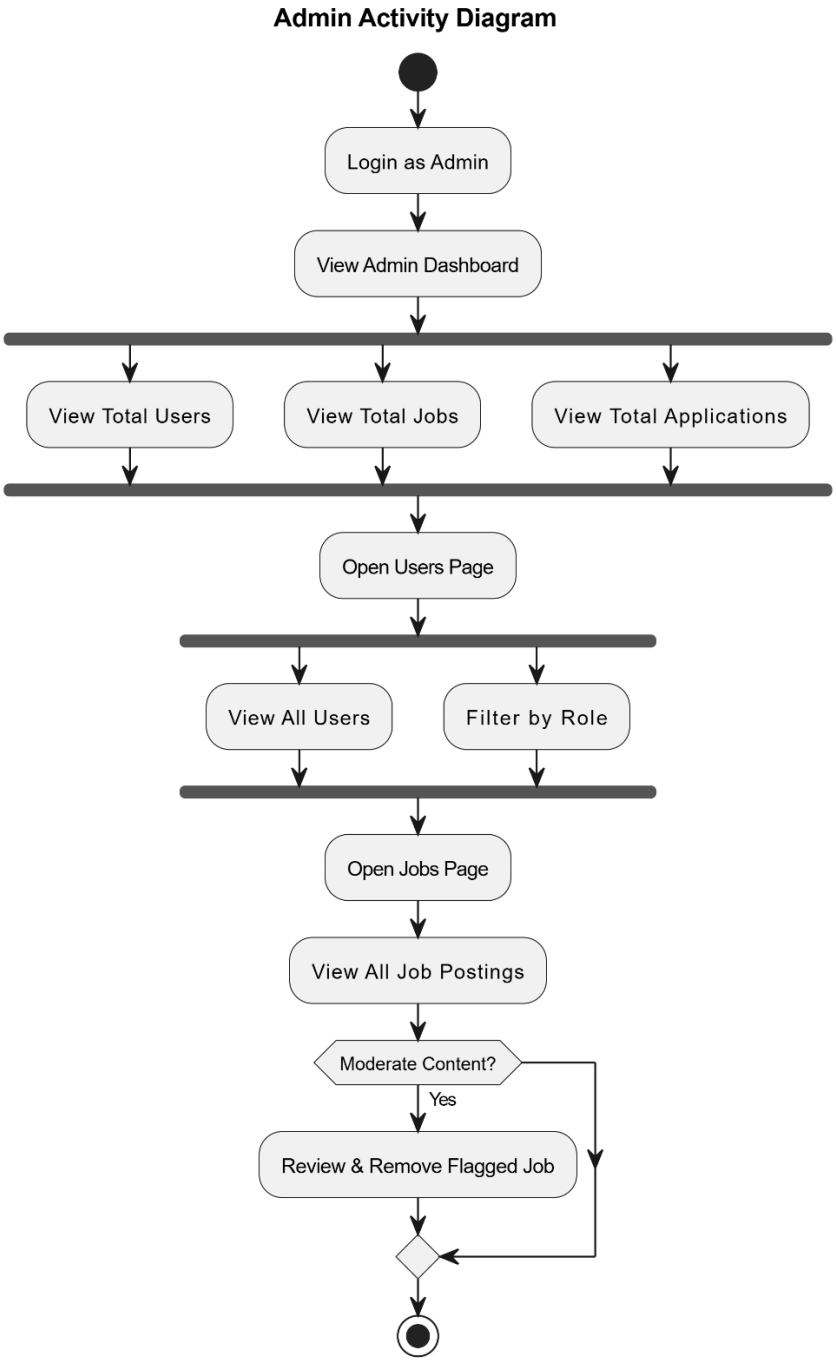


Fig 4.3.3 Admin Activity

4.4 Sequence Diagram

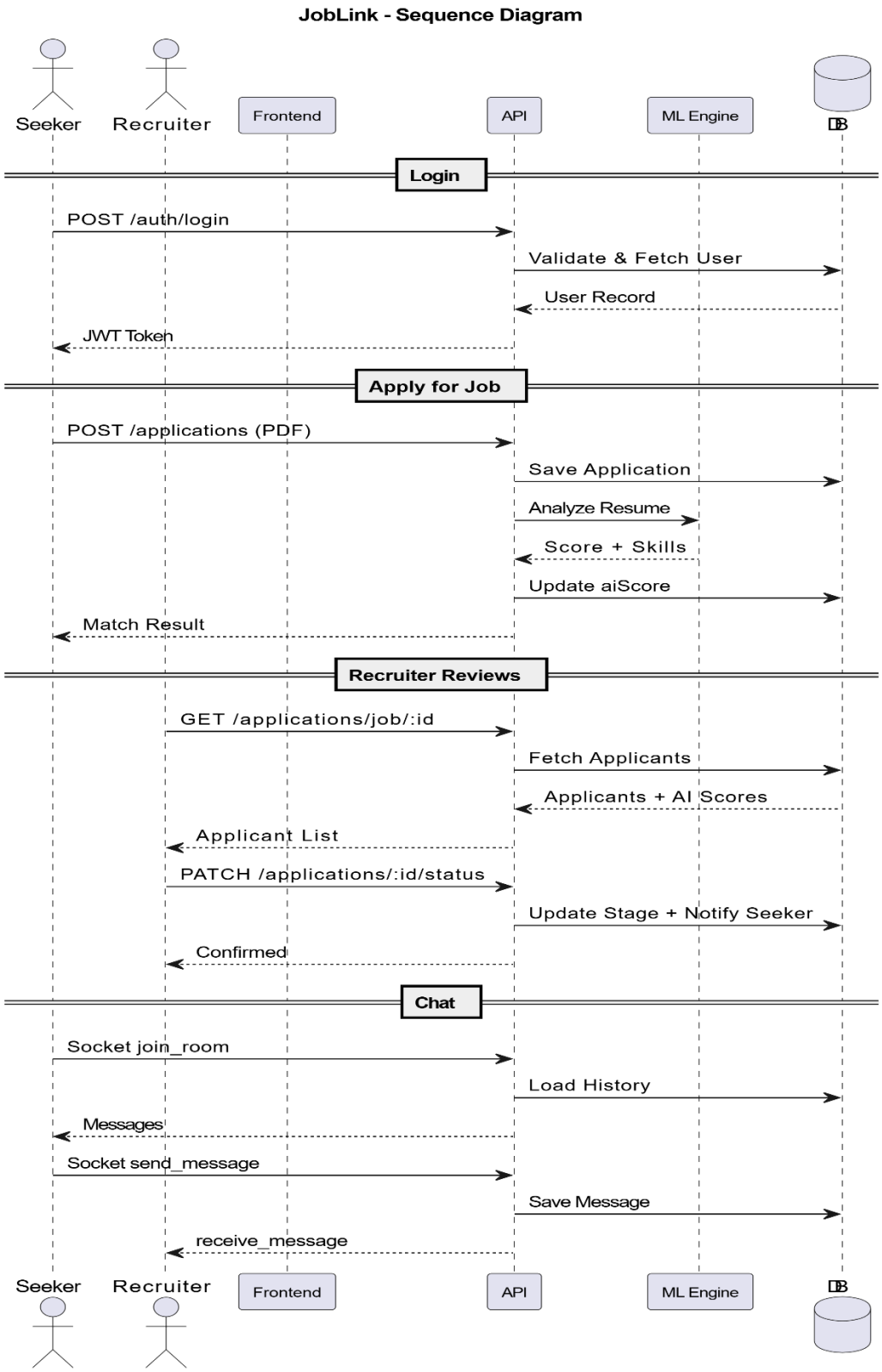


Fig 4.4 Sequence Activity

4.5 Class Diagram Diagram

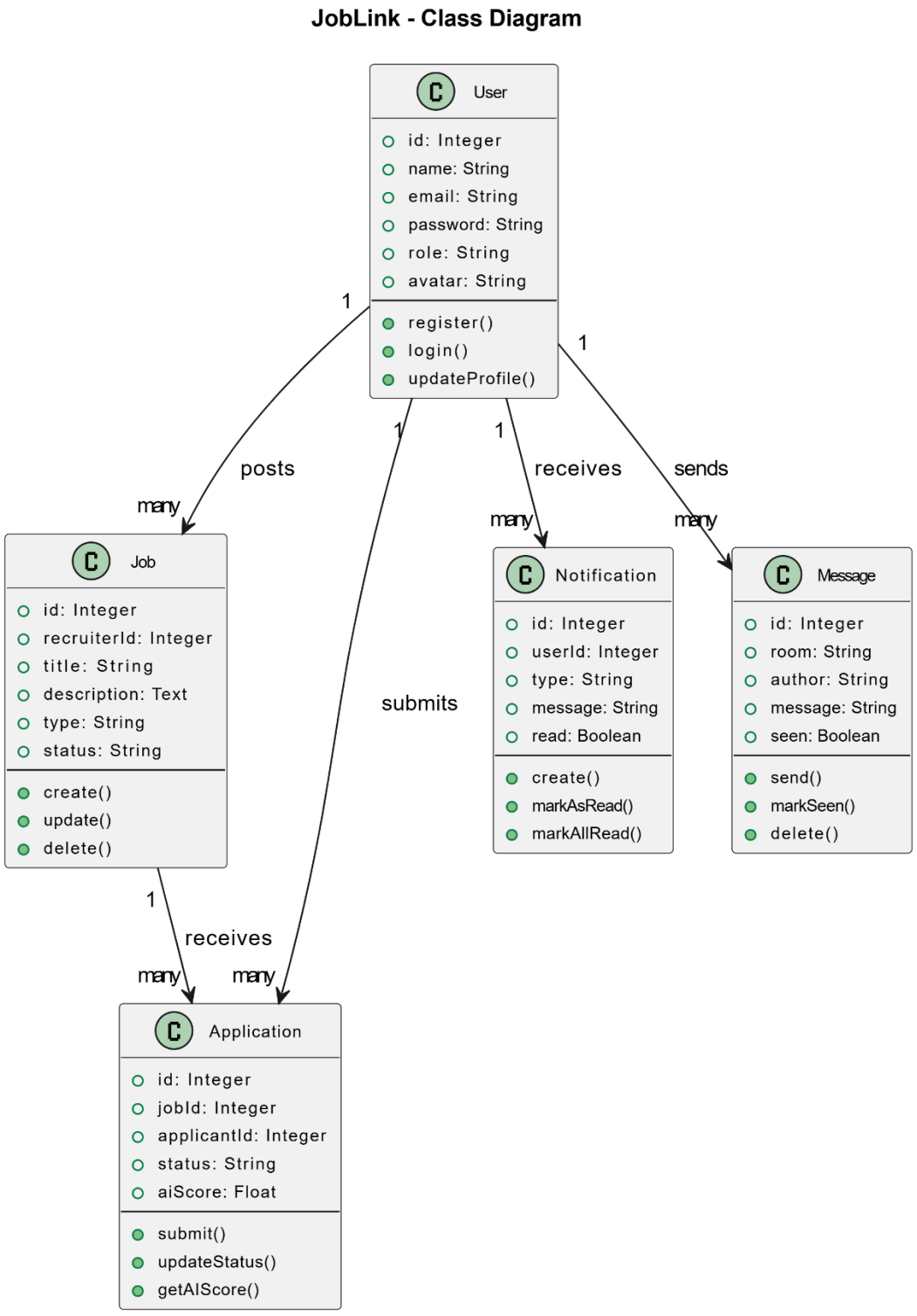


Fig 4.5 Class Diagram

5. TEST CASES

TEST CASES

5.1 User Management

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-UM-001	User Management	Register new seeker account	Valid email, internet connection	Enter name, email, password, role=Seeker → Click Register	Account created, JWT generated, redirected to onboarding
TC-UM-002	User Management	Register with existing email	Email already registered	Enter existing email → Click Register	Error: "User with this email already exists"
TC-UM-003	User Management	Login with valid credentials	Registered user exists	Enter correct email & password → Click Login	Login successful, JWT issued, redirected to dashboard
TC-UM-004	User Management	Access protected route without token	User not logged in	Try accessing dashboard URL directly	Redirected to login page

Table 5.1 User Management

5.2 Job Seeker Test Cases

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-JS-001	Job Seeker	Browse all jobs	Logged in as Seeker	Navigate to Jobs page	All active jobs displayed
TC-JS-002	Job Seeker	Apply with valid PDF resume	Job exists	Open job → Click Apply → Upload PDF → Submit	Application stored, AI score generated
TC-JS-003	Job Seeker	Apply with non-PDF resume	Job exists	Upload .doc file → Submit	Error: “Only PDF allowed”
TC-JS-004	Job Seeker	Duplicate application attempt	Already applied	Click Apply again	Error: “Already applied for this job”

Table 5.2 Job Seeker Test Cases

5.3 Recruiter Test Cases

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-R-001	Recruiter	Post new job	Recruiter logged in	Fill job form → Click Post Job	Job saved and visible to seekers
TC-R-002	Recruiter	Update application stage	Applications exist	Change stage to Interviewed	Status updated, notification sent
TC-R-003	Recruiter	Delete job posting	Job exists	Click Delete Job	Job removed from database

Table 5.3 Real-Time Synchronization

5.4 AI Engine Test Cases

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-AI-001	AI Engine	Hybrid ATS score calculation	Valid resume & job description	Submit resume for application	Score calculated (Semantic + Skill + Domain)
TC-AI-002	AI Engine	Fallback scoring when ML unavailable	ML Engine offline	Submit application	Keyword-based score generated
TC-AI-003	AI Engine	Smart Match role suggestion	Valid resume	Upload resume in Smart Match	Top 5 roles displayed

Table 5.4 Contact Management

5.5 Chat System Test Cases

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-CHAT-001	Chat	Send message	Both users logged in	Open chat → Send message	Message stored and delivered
TC-CHAT-002	Chat	Read receipt update	Unread messages exist	Open conversation	Messages marked as read
TC-CHAT-003	Chat	Soft delete message	Message exists	Delete own message	Message marked deleted

Table 5.5 Admin Monitoring

5.6 Admin Test Cases

Test Case ID	Module	Test Scenario	Preconditions	Test Steps	Expected Result
TC-ADMIN-001	Admin	View all users	Admin logged in	Navigate to Users page	All registered users displayed
TC-ADMIN-002	Admin	Filter users by role	Users exist	Select role filter	Filtered list displayed
TC-ADMIN-003	Admin	View all job postings	Jobs exist	Navigate to Jobs page	All job postings displayed

Table **5.5 Admin Monitoring**

6. INPUT DESGIN AND OUTPUT DESIGN

INPUT DESIGN AND OUTPUT DESIGN

Input and Output Design is a critical part of system development as it ensures that data entered into the system is accurate, validated, secure, and meaningful, while outputs are clear, structured, and useful for decision-making. In the JOBLINK platform, careful attention has been given to designing user-friendly input forms and informative output displays for all user roles including Seeker, Recruiter, and Admin.

Effective input design minimizes user errors and improves data integrity, whereas effective output design enhances usability, clarity, and system transparency.

6.1 Input Design

Input design focuses on how users enter data into the system. The primary objective is to ensure correctness, validation, security, and ease of use.

6.1.1 User Registration Input

The registration form collects the following inputs:

- Name
- Email
- Password
- Role (Seeker / Recruiter)
- Company Name (Required only for Recruiter)

Validation rules include:

- Email format validation and uniqueness check.
- Password minimum length enforcement.
- Mandatory fields validation.
- Password hashing using bcrypt before storage.

Error messages are displayed instantly for invalid or missing inputs, ensuring smooth user experience.

6.1.2 Login Input

The login form accepts:

- Email
- Password

The system verifies credentials against stored hashed passwords. If authentication fails, appropriate error messages are displayed without revealing sensitive information.

6.1.3 Profile Creation and Update Input

Job seekers can input:

- Headline
- Location
- About section
- Skills
- Education
- Resume (PDF upload)
- Profile Picture

File upload inputs are validated to accept only PDF format for resumes. File size restrictions are enforced to maintain performance and security.

Recruiters can update company information and profile details similarly.

6.1.4 Job Posting Input (Recruiter)

Recruiters enter detailed job information including:

- Job Title
- Description
- Location
- Salary
- Employment Type
- Work Mode

- Experience Level
- Industry
- Application Deadline
- Requirements (Tag-based input)
- Responsibilities (Tag-based input)
- Benefits (Tag-based input)

All mandatory fields are validated before submission. The system prevents incomplete job postings from being saved.

6.1.5 Job Application Input

When applying for a job, the seeker uploads a resume in PDF format. The system validates:

- File type (PDF only)
- Duplicate application prevention
- Job availability status

The resume text is extracted and sent to the AI Engine for scoring.

6.1.6 Resume Analyzer Input

The Resume Analyzer module requires:

- Resume PDF upload
- Pasted Job Description text

The system processes both inputs and sends them to the AI Engine without creating a formal application record.

6.1.7 Smart Match Input

The Smart Match feature requires only:

- Resume PDF upload

The resume is analyzed to generate top five recommended job roles.

6.1.8 Chat Message Input

The chat module allows users to:

- Enter text messages
- Send messages in real-time

The system validates non-empty messages and stores them in the database with timestamps.

6.2 Output Design

Output design ensures that information generated by the system is presented clearly, accurately, and meaningfully.

6.2.1 Dashboard Output

Each user role has a customized dashboard:

- Seeker Dashboard: Displays application summary, match scores, and recent notifications.
- Recruiter Dashboard: Displays posted jobs, applicant counts, and quick action options.
- Admin Dashboard: Displays platform-wide statistics including total users and total job postings.

6.2.2 Job Listings Output

Jobs are displayed in structured card format containing:

- Job Title
- Company Name
- Location
- Employment Type
- Short Description
- Apply Button

Filters dynamically update the output without page reload.

6.2.3 AI Match Score Output

After job application submission or resume analysis, the system displays:

- Overall ATS Score
- Match Level (High / Medium / Low)
- Semantic Score
- Skill Overlap Score
- Domain Boost Contribution
- Matched Skills
- Missing Skills
- Personalized Improvement Recommendations

This output enhances transparency and helps candidates improve their resumes.

6.2.4 Application Status Output

The My Applications page displays:

- Job Title
- Company Name
- AI Match Score
- Current Pipeline Stage (Applied / Under Review / Interviewed / Rejected / Accepted)
- Application Date

Status updates are reflected in real time.

6.2.5 Notification Output

Notifications are displayed via a notification bell icon showing unread count. Each notification contains:

- Title
- Message
- Timestamp
- Deep Link to related page

Clicking a notification marks it as read and redirects the user.

6.2.6 Chat Output

Chat output includes:

- Conversation history (last 100 messages)
- Message timestamps
- Read receipts
- Soft-delete indicators
- User avatars

Messages are displayed instantly using real-time broadcasting.

6.2.7 Admin Output

The Admin panel outputs:

- List of all registered users
- Role-based filtering
- List of all job postings
- Aggregate platform statistics

This output supports platform monitoring and moderation.

6.3 Design Principles Followed

The input and output design of JOBLINK follows key principles:

- Simplicity and clarity
- Validation at both frontend and backend
- Consistent layout across modules
- Role-specific interface customization
- Secure handling of sensitive information
- Real-time feedback mechanisms

7. SYSTEM IMPLEMENTATION

SYSTEM IMPLEMENTATION

System Implementation is the phase in which the designed system is transformed into a fully functional software product. In the JOBLINK platform, implementation involved developing the frontend interface, backend API services, AI-based resume evaluation engine, real-time communication server, and database schema. The system was implemented using modern full-stack technologies following a modular and scalable architecture.

The implementation phase ensured that all functional and non-functional requirements defined in earlier chapters were translated into working modules integrated seamlessly into a unified platform.

7.1 Implementation Environment

The JOBLINK system was implemented using the following technology stack:

Frontend Layer:

- React.js (with Vite for development and production bundling)
- Axios for API communication
- React Router for navigation
- React Hot Toast for notifications

Backend Layer:

- Node.js
- Express.js framework
- JWT for authentication
- bcryptjs for password hashing
- Multer middleware for file uploads
- pdf-parse for resume text extraction

AI Engine:

- Python
- FastAPI framework

- Sentence-BERT model for semantic similarity
- Hybrid scoring algorithm implementation

Database:

- PostgreSQL relational database

Real-Time Communication:

- Socket.IO server running on port 3001

This multi-tier architecture ensures separation of concerns and allows independent scaling of components.

7.2 Frontend Implementation

The frontend was implemented using React.js with a component-based architecture. The application is divided into role-specific layouts:

- SeekerLayout
- RecruiterLayout
- AdminLayout

Each layout provides contextual navigation and dashboard access specific to the user role.

Routing was implemented using protected route wrappers to ensure that only authenticated users with appropriate roles can access specific pages. JWT tokens are stored securely and attached to API requests using the Authorization header with the Bearer scheme.

Forms such as registration, job posting, resume upload, and profile update include client-side validation to enhance usability and reduce invalid submissions.

Dynamic filtering for job listings is handled using client-side state management. AI score results, notifications, and chat messages are displayed in real time.

7.3 Backend Implementation

The backend was implemented using Node.js with the Express framework. The architecture follows a modular directory structure separating:

- Routes
- Controllers
- Models
- Middleware
- Utility services

7.3.1 Authentication and Authorization

User authentication is handled using JWT tokens. During registration, passwords are hashed using bcrypt with 10 salt rounds before being stored in PostgreSQL. During login, the hashed password is compared securely.

Role-based access control is implemented through middleware functions that validate both authentication and user role before granting access to protected routes.

7.3.2 Job Management APIs

Recruiters can create, update, and delete job postings. Each job record is stored in the Jobs table with fields including title, description, salary, location, requirements, responsibilities, and benefits.

Seekers can retrieve active job listings through GET endpoints. Duplicate application prevention logic ensures that a seeker cannot apply to the same job more than once.

7.3.3 Resume Processing

When a resume is uploaded, the Multer middleware validates file type and size. The resume file is temporarily stored and processed using pdf-parse to extract textual content.

The extracted text is forwarded to the AI Engine via HTTP POST request for semantic evaluation.

7.4 AI Engine Implementation

The AI Engine was implemented as a separate Python FastAPI service running independently from the main backend.

At startup, the Sentence-BERT model is loaded into memory to avoid repeated initialization overhead. Job role embeddings are pre-computed to ensure faster inference.

The hybrid ATS scoring algorithm combines:

- 60% semantic similarity using SBERT embeddings
- 30% skill overlap comparison
- 10% domain-specific boost

The engine returns structured JSON responses containing overall score, component scores, matched skills, missing skills, and improvement suggestions.

A fallback keyword-based scoring mechanism is implemented on the Node.js side to ensure uninterrupted application processing if the AI service becomes unavailable.

7.5 Database Implementation

PostgreSQL was used as the relational database management system. The primary tables include:

- Users
- Jobs
- Applications
- Notifications
- Messages

Primary keys and foreign keys ensure referential integrity. Indexing is applied to frequently queried fields such as userId and jobId for performance optimization.

Timestamps (createdAt, updatedAt) are automatically maintained for tracking system activity.

7.6 Real-Time Chat Implementation

The real-time communication module was implemented using Socket.IO running on port 3001. When two users initiate a conversation, a deterministic room ID is generated by sorting and combining their user IDs.

The server handles:

- join_room event
- send_message event
- mark_read event
- delete_message event

Messages are stored in the Messages table with fields such as room, author, message, seen, deleted, and timestamp.

Each new message also generates a notification entry in the Notifications table for the recipient.

7.7 Notification System Implementation

Notifications are generated during:

- Application stage updates
- New chat messages

Each notification record contains type, title, message, relatedLink, and read status. The frontend displays unread count in a notification bell icon.

Clicking a notification sends a PATCH request to mark it as read.

7.8 Error Handling and Logging

All backend operations are wrapped in try-catch blocks to prevent unhandled exceptions. Meaningful HTTP status codes are returned for success and failure cases.

If the ML Engine is unavailable, the system logs the error and automatically applies fallback scoring without affecting the user experience.

If the chat server fails to initialize due to port conflicts, the backend logs a warning without crashing the main API process.

7.9 Deployment Considerations

The system is designed to support deployment in a production environment with:

- Environment variables stored in `.env` files
- Stateless JWT authentication enabling horizontal scaling
- Independent scaling of ML Engine, Chat Server, and REST API

This microservices-inspired design ensures scalability and maintainability as platform usage grows.

8. AI INTEGRATION(MODULE DESCRIPTION /APIs USED)

AI INTEGRATION (MODULE DESCRIPTION / APIs USED)

Artificial Intelligence plays a central role in the JOBLINK platform. The AI module is responsible for resume evaluation, semantic job matching, smart job role prediction, and generating personalized improvement suggestions. The system uses Natural Language Processing (NLP) and Machine Learning techniques to simulate Applicant Tracking System (ATS) style evaluation.

The AI module is implemented as a separate microservice to ensure scalability, flexibility, and independent deployment.

8.1 AI Module Architecture

The AI Engine is developed using:

- Python
- FastAPI framework
- Sentence-BERT (SBERT) pre-trained transformer model
- Scikit-learn utilities for similarity calculations

The module runs independently and communicates with the Node.js backend using REST APIs. When a resume is uploaded, the backend extracts the resume text and sends it to the AI Engine along with the job description.

The AI Engine processes the data and returns a structured JSON response containing scoring metrics and suggestions.

8.2 Resume Evaluation Module

This module evaluates how well a candidate's resume matches a given job description.

Working Process:

1. Resume PDF is uploaded by the user.
2. Text is extracted using pdf parsing.
3. Extracted text and job description are sent to the AI Engine.

4. SBERT converts both texts into vector embeddings.
5. Cosine similarity is calculated between vectors.
6. Skill overlap and domain relevance are computed.
7. Final ATS score is generated.

Hybrid Scoring Formula:

The final ATS Score is calculated using a weighted hybrid model:

- 60% Semantic Similarity Score
- 30% Skill Overlap Score
- 10% Domain Boost Score

This ensures both contextual meaning and keyword relevance are considered.

8.3 Smart Match Module

The Smart Match feature predicts the most suitable job roles for a candidate based solely on their resume.

Working:

- Resume text is embedded using SBERT.
- Precomputed embeddings of predefined job roles are stored in memory.
- Cosine similarity is calculated between resume embedding and role embeddings.
- Top 5 most similar roles are returned with match percentages.

This module helps users understand which roles best align with their profile.

8.4 Resume Analyzer (Without Application)

This module allows candidates to check resume compatibility without formally applying for a job.

Inputs:

- Resume PDF

- Job Description text

Output:

- Overall ATS Score
- Semantic Similarity Score
- Skill Overlap Score
- Missing Skills
- Improvement Suggestions

This acts as a practice evaluation tool for candidates.

8.5 APIs Used in AI Integration

The AI Engine exposes REST APIs that communicate with the main backend.

1. Resume Match API

Endpoint:

```
POST /match
```

Input JSON:

- resume_text
- job_description
- job_title

Output JSON:

- overall_score
- semantic_score
- skill_score
- domain_boost
- matched_skills
- missing_skills
- suggestions

This API is triggered when a user applies for a job.

2. Smart Match API

Endpoint:

```
POST /smart-match
```

Input JSON:

- resume_text

Output JSON:

- top_roles (list of predicted job roles)
- similarity_scores

This API is used in the Smart Match feature.

3. Health Check API

Endpoint:

```
GET /health
```

Used by the backend to verify whether the AI Engine is active. If unavailable, the system activates fallback scoring.

8.6 Fallback Scoring Mechanism

To ensure system reliability, a keyword-based fallback scoring algorithm is implemented in the backend.

If the AI Engine fails:

- Resume and job description are tokenized.
- Keyword overlap percentage is calculated.
- A simplified score is generated.

This guarantees uninterrupted job application processing even during AI service downtime.

8.7 Advantages of AI Integration

- Improves fairness and objectivity in screening
- Provides transparent scoring breakdown
- Helps candidates improve resumes
- Reduces recruiter manual screening effort
- Enables intelligent job recommendations

8.8 Security and Performance Considerations

- Resume files are temporarily processed and not permanently stored in the AI server.
- Preloaded embeddings reduce response time.
- Microservice separation ensures better scalability.
- Error handling prevents system crashes if AI service fails.

The AI integration in JOBLINK transforms the platform from a traditional job portal into an intelligent career guidance and recruitment support system.

9. FUTURE ENHANCEMENTS

FUTURE ENHANCEMENTS

The JOBLINK platform has been designed with scalability and extensibility in mind. Although the current system provides intelligent resume evaluation, job matching, and real-time communication, several enhancements can be implemented in future versions to improve functionality, accuracy, and user experience.

9.1 Advanced AI Improvements

The current AI model uses Sentence-BERT with a hybrid scoring algorithm. Future improvements may include:

- Fine-tuning transformer models on recruitment-specific datasets.
- Introducing deep learning-based skill extraction instead of keyword matching.
- AI-powered resume rewriting suggestions.
- Interview question generation based on resume and job description.
- AI-driven candidate ranking dashboard for recruiters.

These improvements would increase evaluation accuracy and automation efficiency.

9.2 Video Interview Integration

A built-in video interview module can be added to enable:

- Live video interviews within the platform.
- AI-based facial expression and sentiment analysis.
- Automated interview performance scoring.

This would reduce dependency on third-party tools and streamline hiring workflows.

9.3 Mobile Application Development

Developing Android and iOS mobile applications would enhance accessibility. Push notifications, instant chat, and real-time updates would improve user engagement and retention.

9.4 Advanced Analytics Dashboard

Future versions can include:

- AI-powered hiring analytics for recruiters.
- Skill demand trend analysis.
- Industry-based job market insights.
- Candidate success prediction models.

These features would make JOBLINK a data-driven recruitment intelligence platform.

9.5 Blockchain-Based Resume Verification

To prevent fake credentials and resume fraud, blockchain-based certificate verification can be integrated. Verified education and experience records would increase trust between candidates and recruiters.

9.6 Multi-Language Support

Currently, the platform operates primarily in English. Future versions can support multiple regional and international languages to expand global accessibility.

9.7 Integration with External Platforms

Integration with platforms such as:

- LinkedIn profile import
- GitHub repository analysis
- Online certification platforms

would allow automatic profile enrichment and better candidate evaluation.

9.8 AI Chatbot Career Assistant

An AI-powered career guidance chatbot could:

- Suggest skill improvements

- Recommend certifications
- Provide interview preparation tips
- Answer career-related queries

This would transform JOBLINK into a comprehensive career development platform.

9.9 Enhanced Security Measures

Future improvements may include:

- Two-Factor Authentication (2FA)
- Advanced encryption standards
- Role-based granular permission control
- Automated anomaly detection for suspicious activities

These measures will strengthen data protection and user trust.

9.10 Cloud-Native Scalable Deployment

Migrating the system to a fully cloud-native architecture using containerization and orchestration tools would allow:

- Auto-scaling during high traffic
- Better performance monitoring
- Reduced infrastructure downtime

The proposed future enhancements aim to transform JOBLINK from an intelligent job portal into a comprehensive AI-powered recruitment and career management ecosystem.

10. CONCLUSION

CONCLUSION

The JOBLINK platform was successfully designed and implemented as an AI-powered job recruitment and career support system. The project integrates modern web technologies with advanced Natural Language Processing techniques to create an intelligent and efficient hiring ecosystem.

The system provides role-based access for Job Seekers, Recruiters, and Administrators, ensuring structured interaction between all stakeholders. Key features such as resume evaluation, hybrid ATS scoring, smart job role prediction, real-time chat, notifications, and job management modules were implemented effectively.

The integration of the AI Engine using Sentence-BERT and semantic similarity analysis significantly enhances traditional recruitment systems by providing transparent and data-driven resume screening. The hybrid scoring model combining semantic similarity, skill overlap, and domain relevance ensures balanced and fair evaluation of candidates.

The modular microservice architecture allows independent scaling of the backend server, AI engine, chat server, and database. This improves maintainability, performance, and future extensibility of the system. Error handling mechanisms and fallback scoring ensure uninterrupted functionality even during service failures.

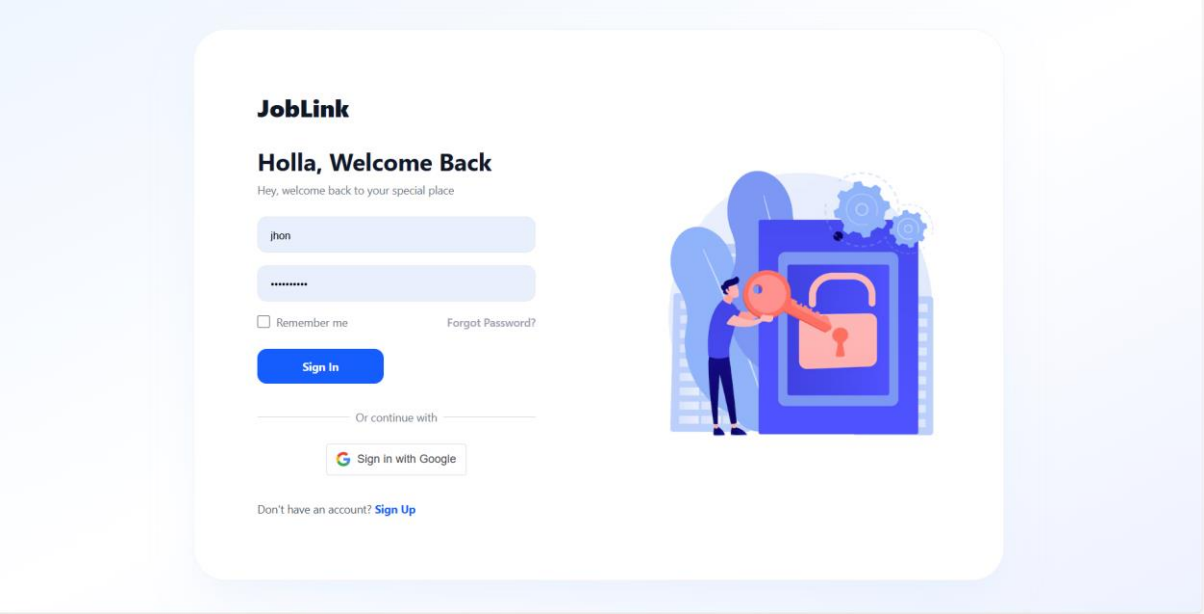
Overall, the project demonstrates the practical application of Artificial Intelligence in solving real-world recruitment challenges. JOBLINK not only simplifies the hiring process for recruiters but also empowers candidates with meaningful feedback and career insights.

The system establishes a strong foundation for future enhancements such as advanced AI analytics, mobile applications, interview automation, and intelligent career assistance, making it a scalable and innovative solution in the field of smart recruitment systems.

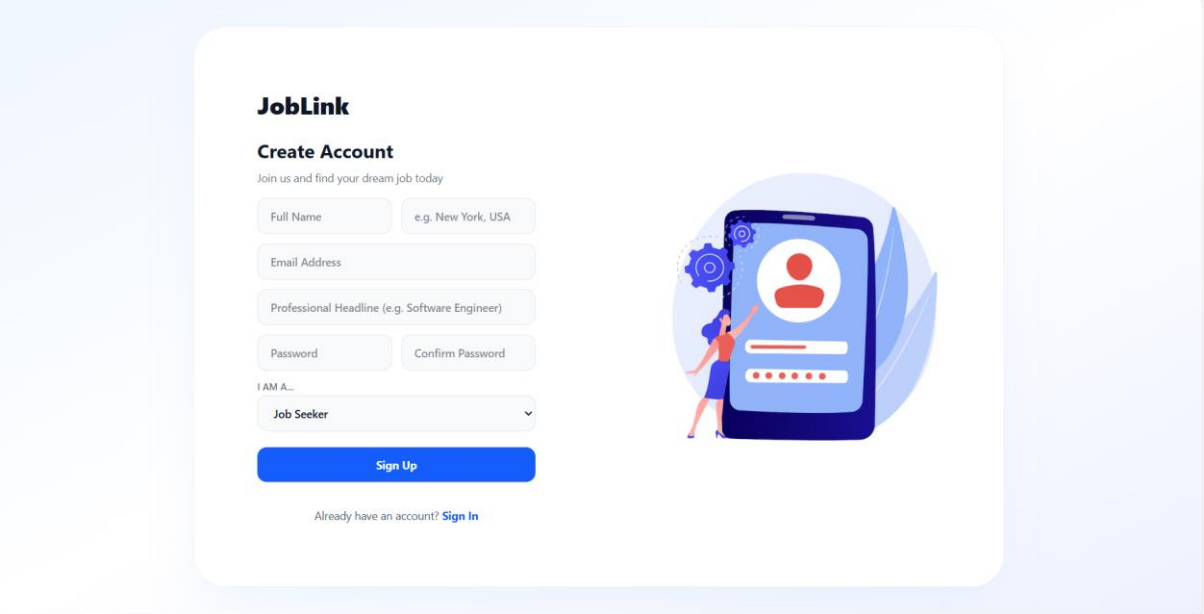
11. ANNEXURE

SCREENSHOTS

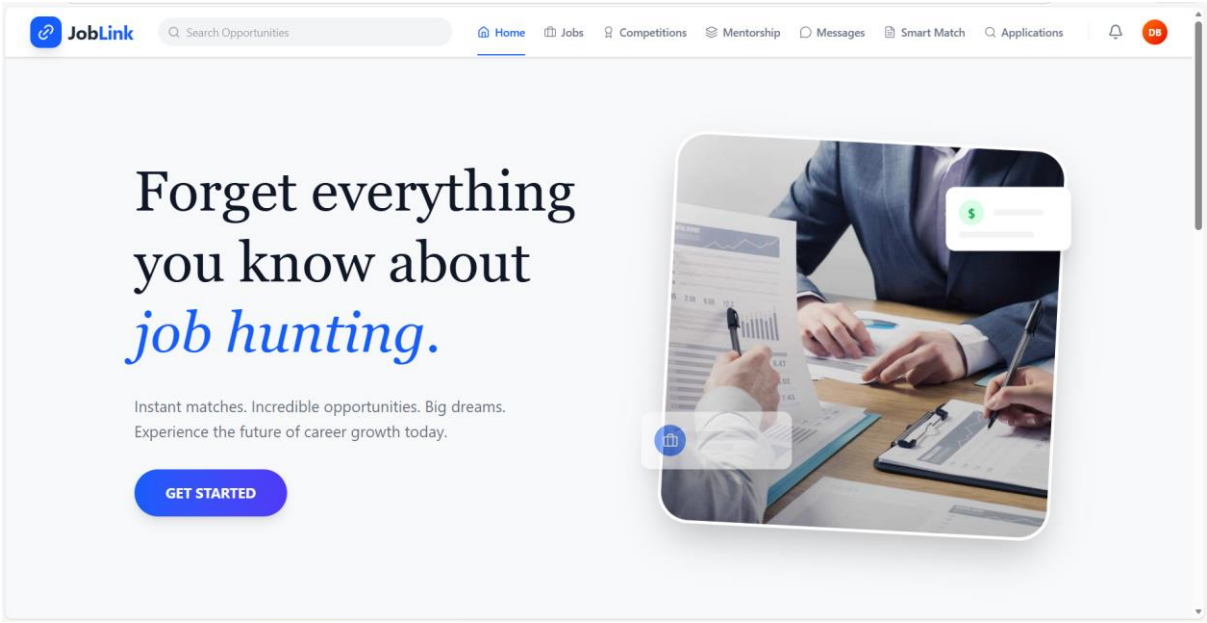
LOGIN PLAYER



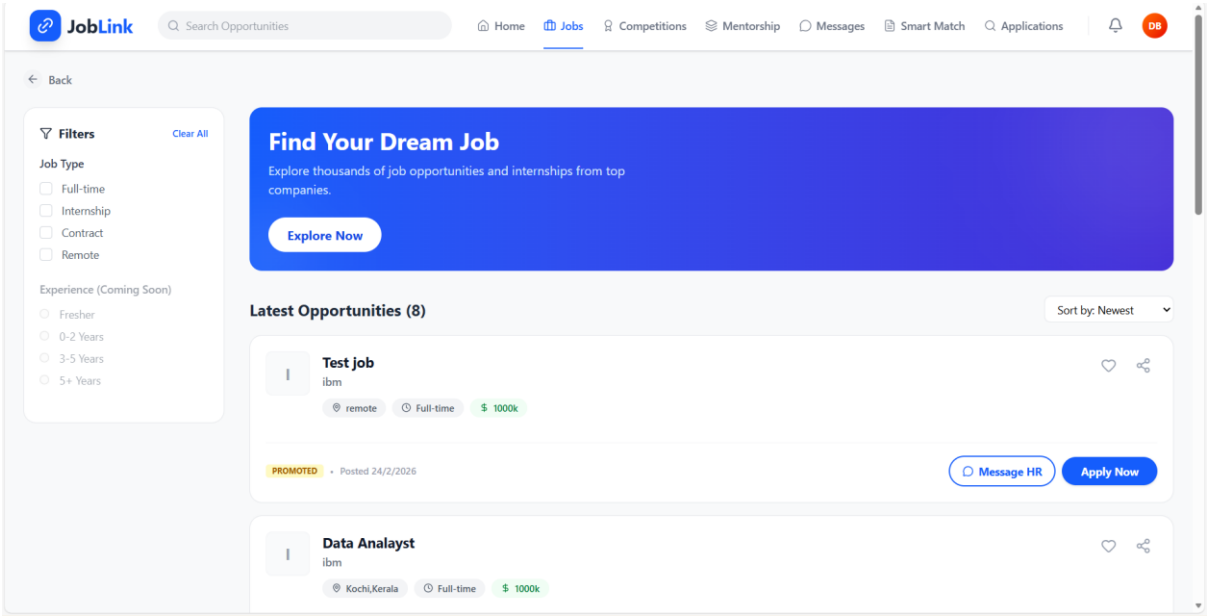
REGISTRATION PAGE



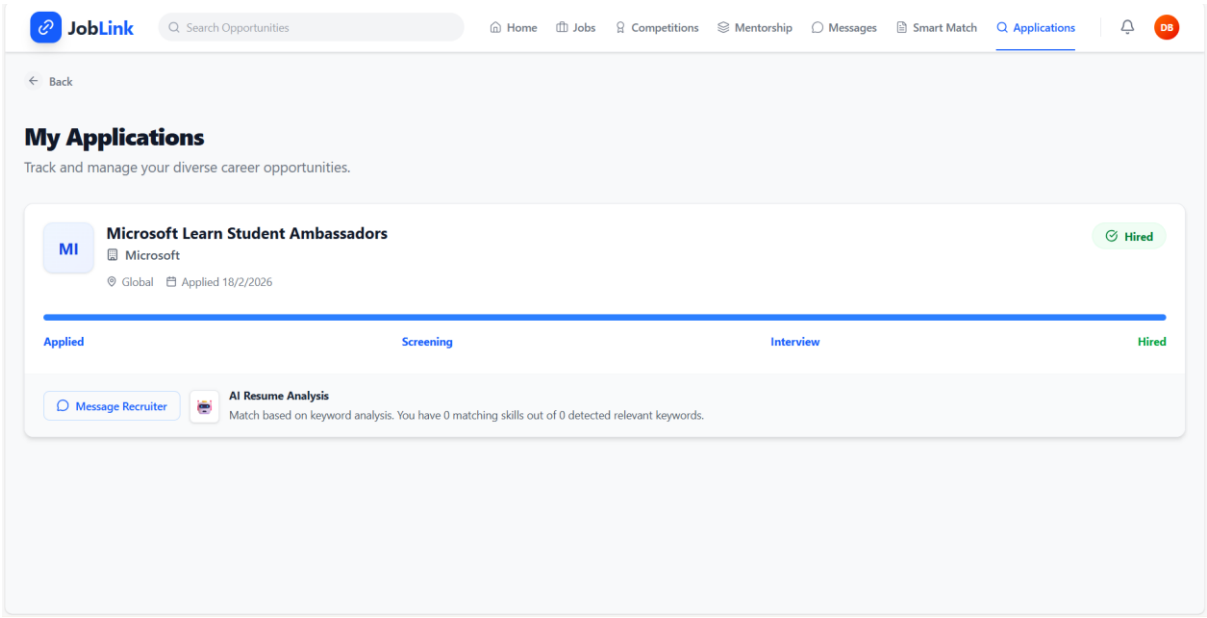
SEEKER HOME DASHBOARD



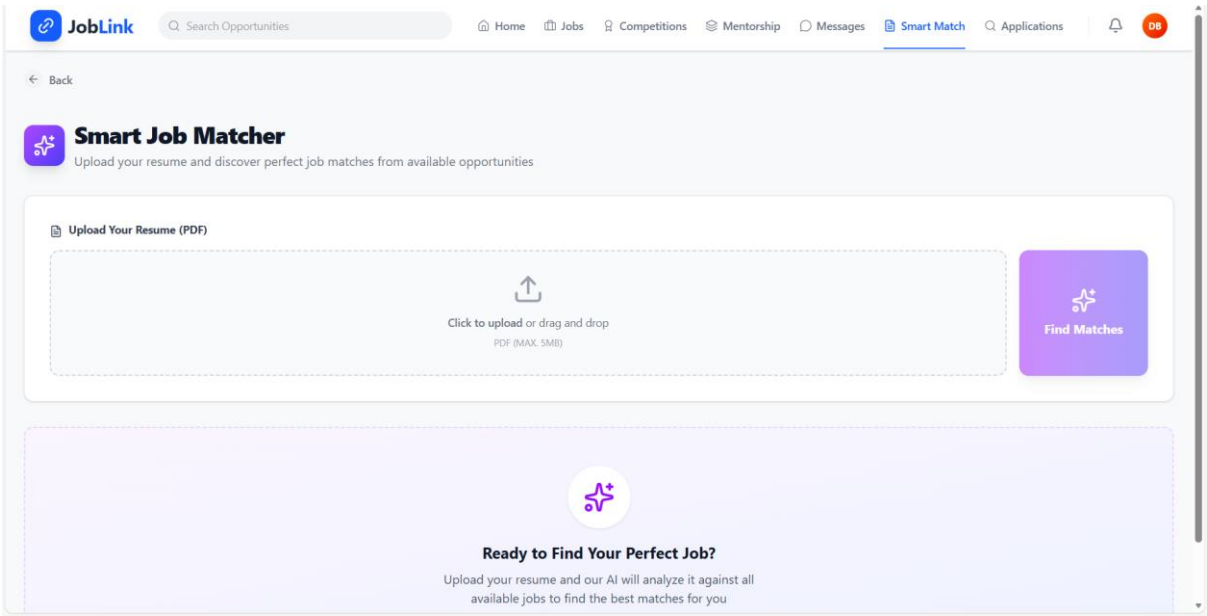
BROWSE JOBS PAGE



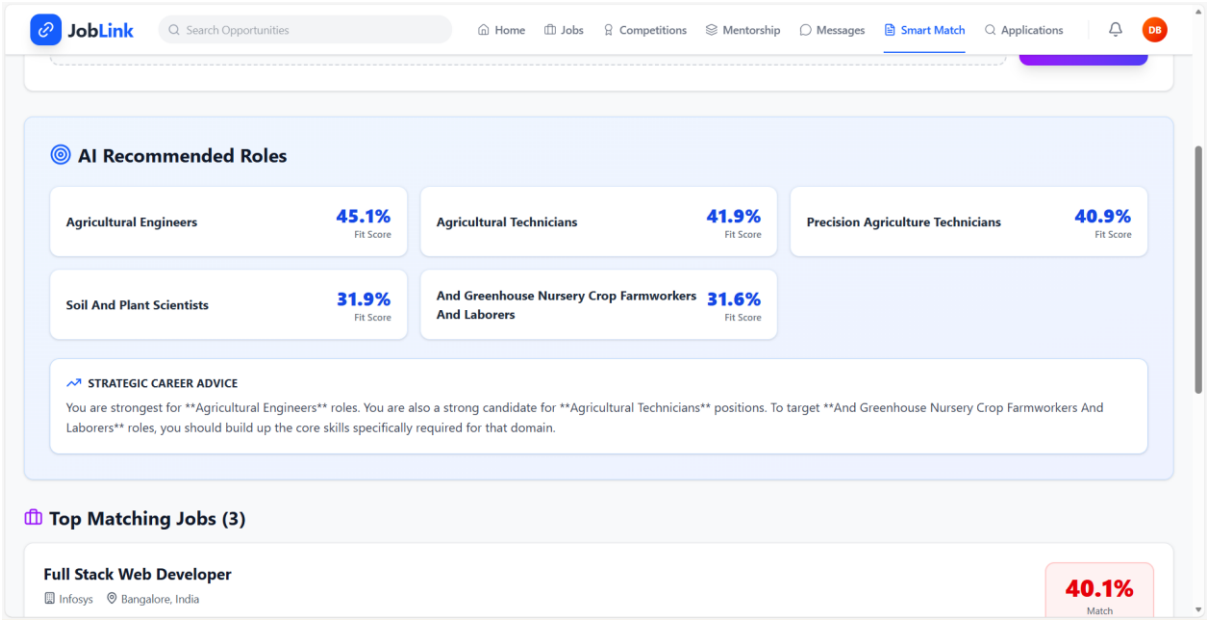
MY APPLICATIONS PAGE



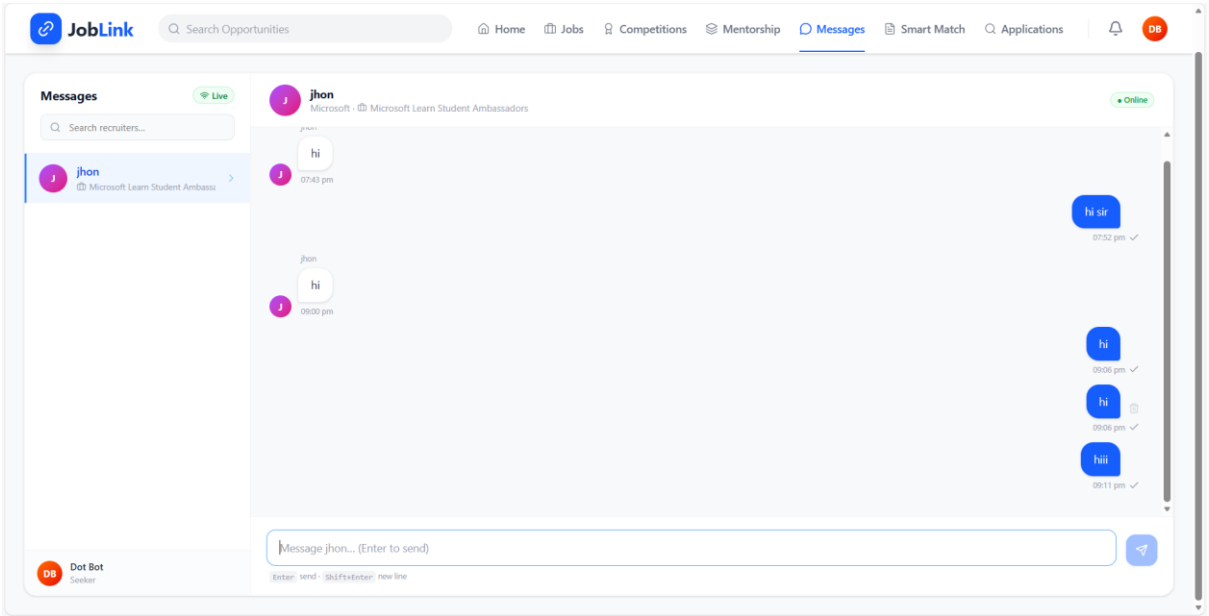
RESUME ANALYZER PAGE



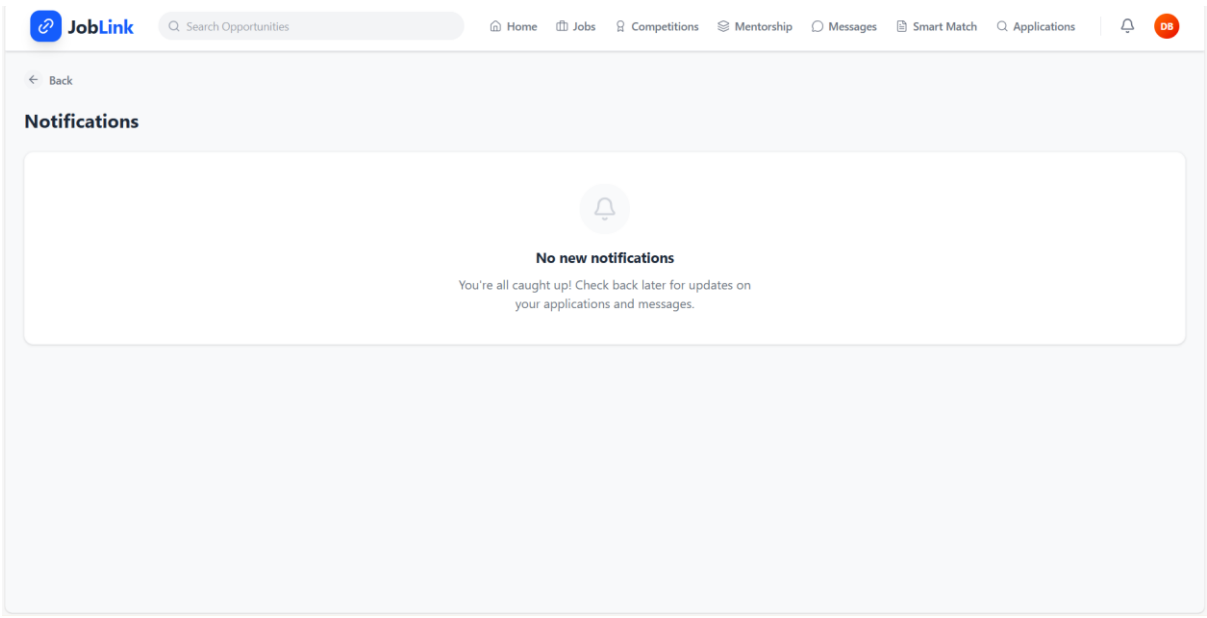
RESUME ANALYZER RESULT



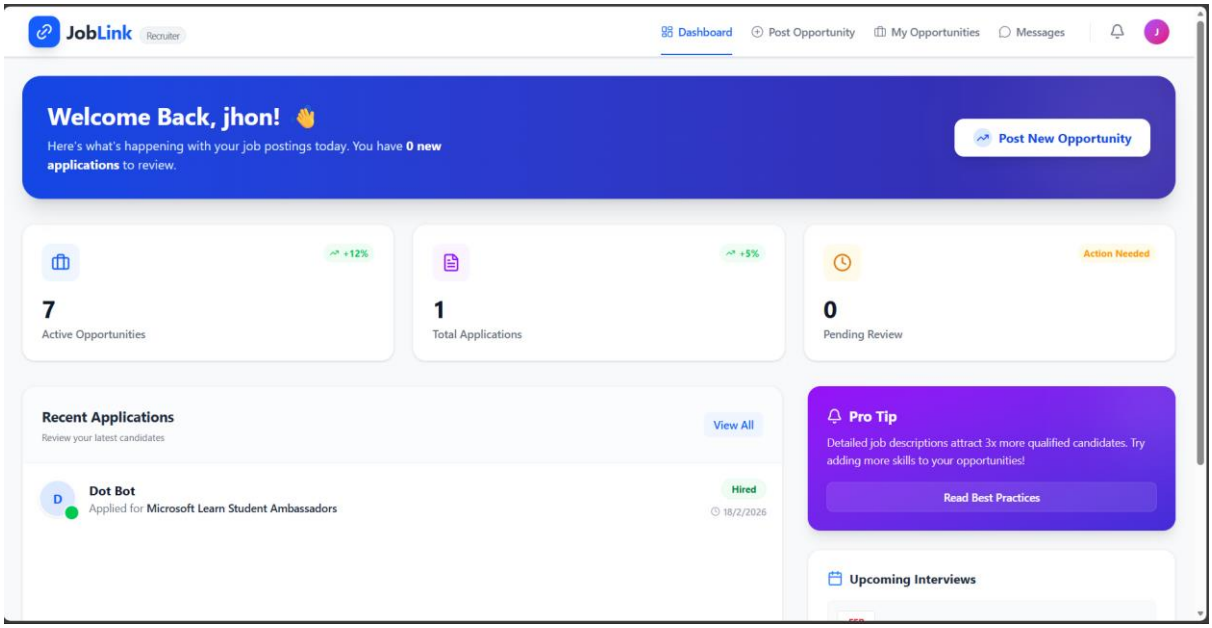
SEEKER MESSAGES / CHAT



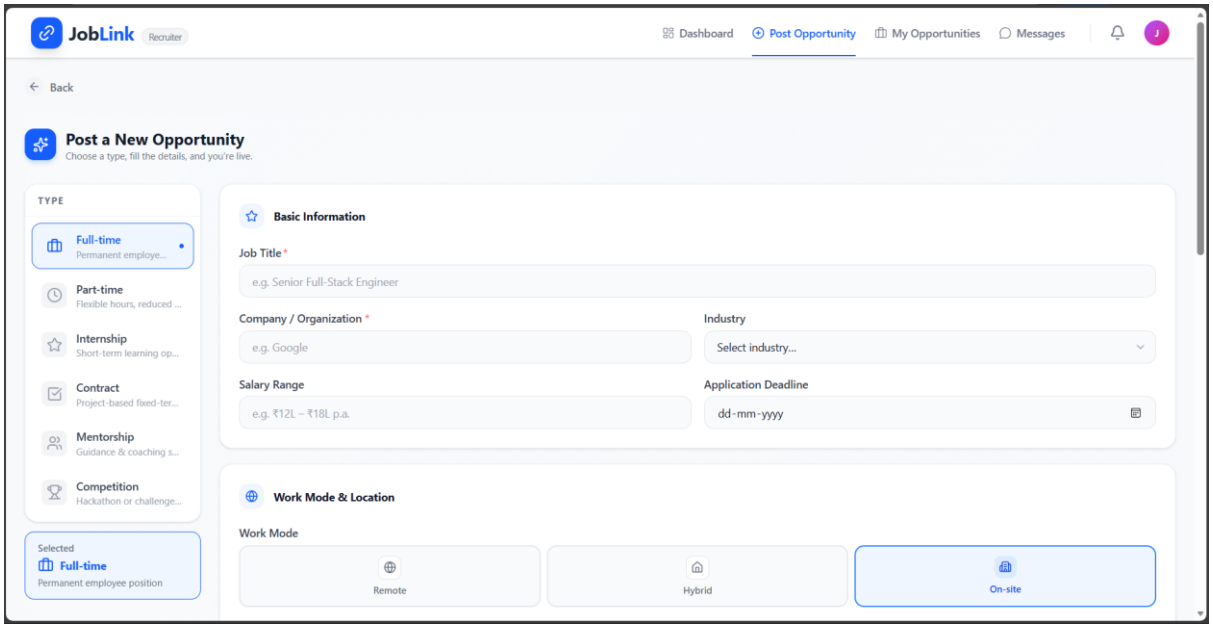
SEEKER NOTIFICATIONS



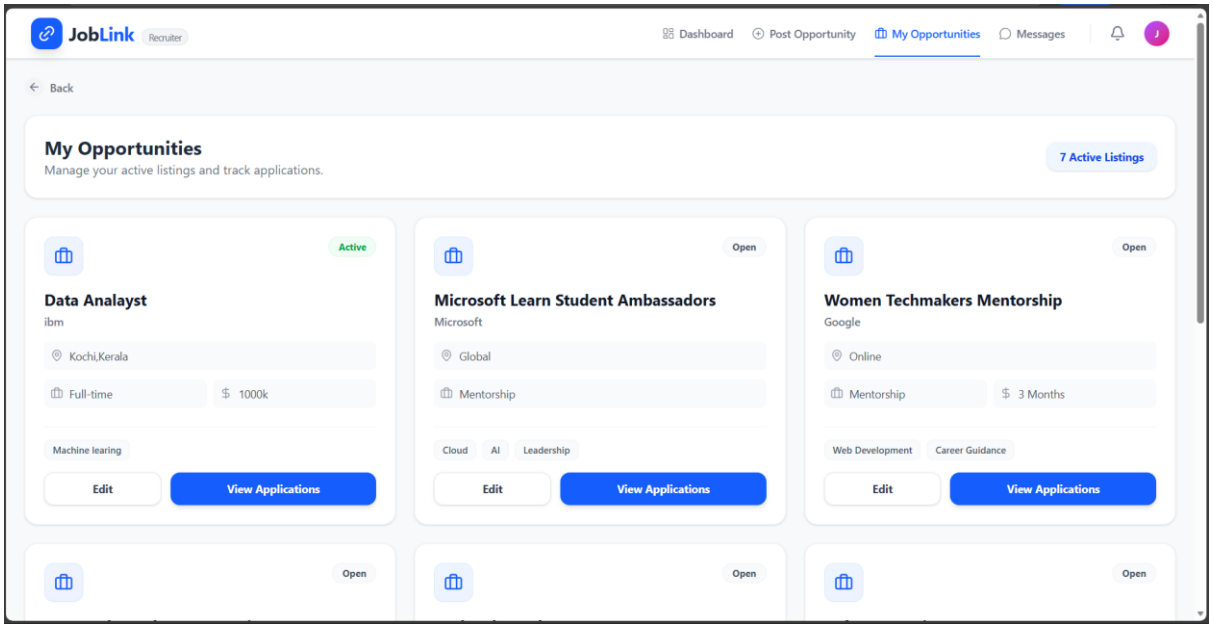
RECRUITER DASHBOARD



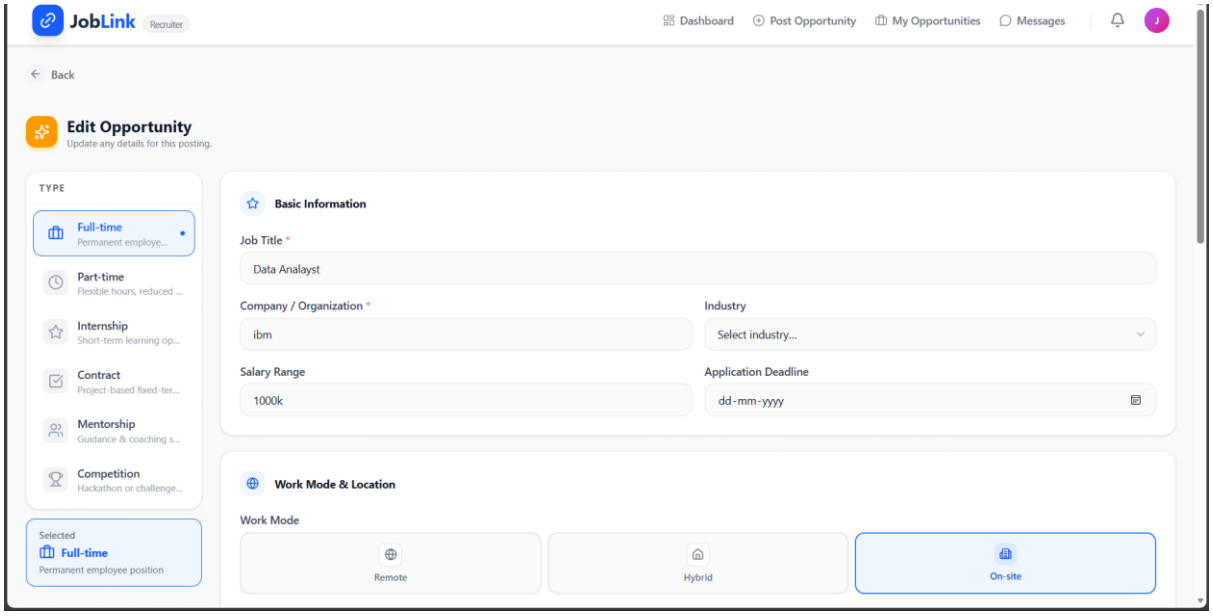
POST JOB PAGE



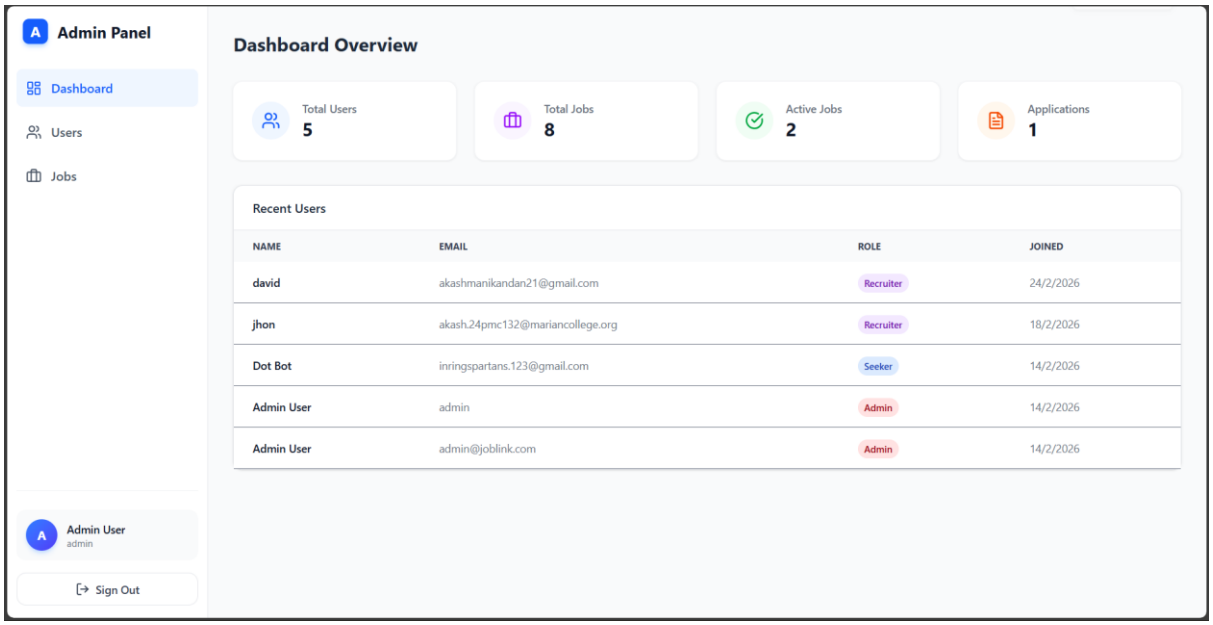
MY JOBS PAGE



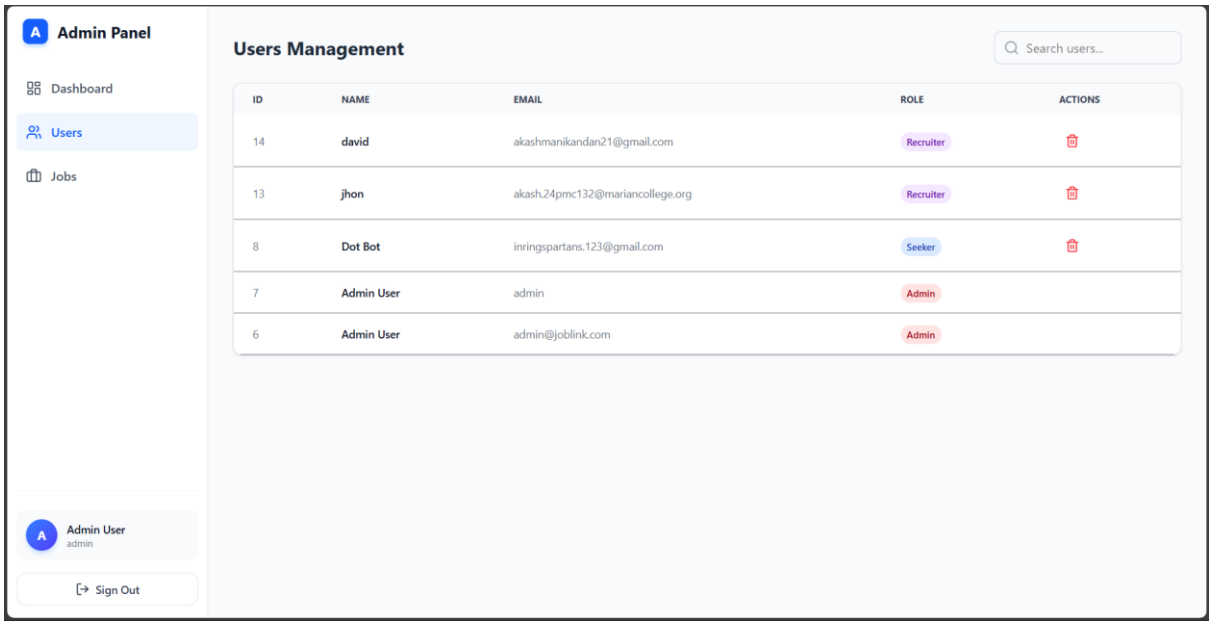
EDIT JOB PAGE



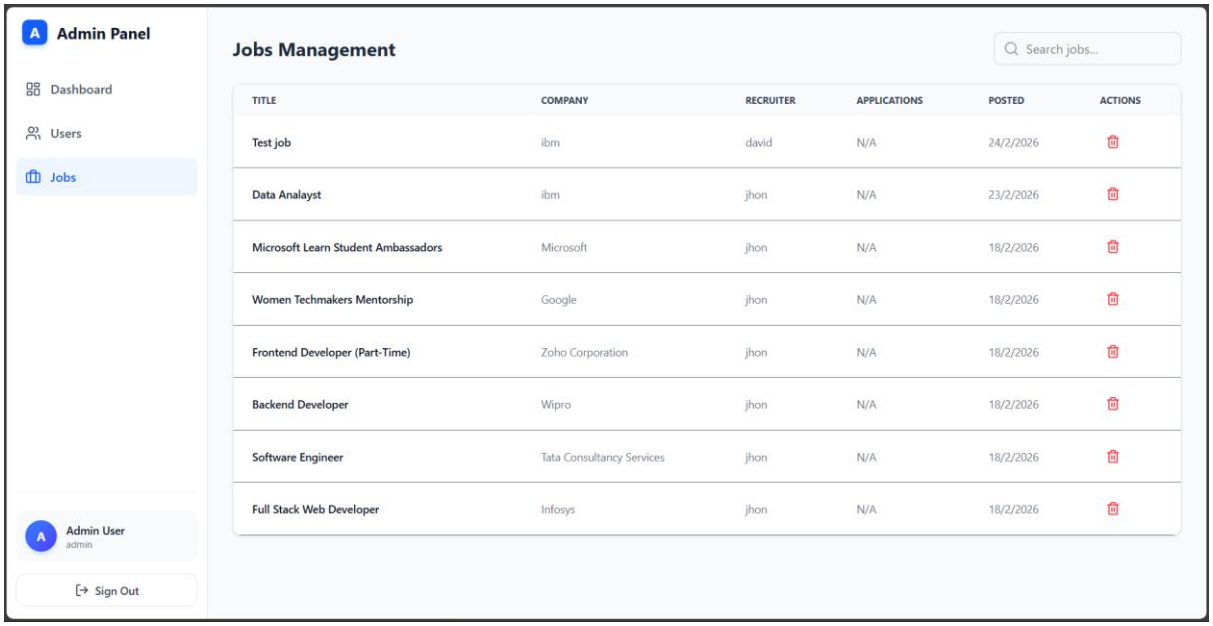
ADMIN DASHBOARD



ADMIN USERS PAGE



ADMIN JOBS PAGE



MCA Student Project Report

Ethical Clearance Form

Department: PG Department of Computer Applications

Academic Year: 2025-26

A. Student & Project Details

1. Student Name : M Akash

2. Admission Number : 2018

3. Programme of Study : MCA

4. Institution Email : akash.24pmc132@mariancollege.org

5. Contact Phone : 6238324918

6. Project/Dissertation Title : JOBLINK

7. Project Modality ☐ Dissertation ☐ Capstone ☐ Internship Project ☐ Other: __

8. Supervisor (Faculty Guide) : Reny Jose

9. DRAC/Departmental Research Advisory Member (if applicable) Dr.Sr.Italia Joseph Maria

10. Industry/External Mentor (if applicable) _____

11. Estimated Start Date :

12. Estimated Completion Date :

B. Project Overview

13. Purpose & Rationale (aim, research problem, expected contribution; attach pages if needed):

JobLink – Intelligent Recruitment Platform with Integrated AI-Powered Applicant Tracking System is a centralized, intelligent, and technology-driven digital platform designed to

streamline the hiring process while ensuring objective and transparent candidate evaluation. The system combines end-to-end recruitment management features with AI-powered mechanisms such as automated resume scoring, real-time pipeline tracking, in-app notifications, and direct recruiter-candidate communication within a unified structured environment. It supports job seekers, recruiters, and administrators, improving coordination, hiring efficiency, and candidate experience throughout the complete recruitment lifecycle.

The project addresses two major challenges in modern recruitment: fragmented hiring management systems and the lack of integrated AI-based candidate evaluation tools. Traditional recruitment platforms primarily focus on job listings and application collection but often neglect automated screening, real-time status transparency, or structured applicant pipeline management. On the other hand, standalone resume screening tools do not integrate with job management workflows, communication systems, or role-based access control. This separation creates evaluation inconsistencies, delayed recruiter-candidate communication, and limited visibility for job seekers into their application progress. JobLink resolves these gaps by unifying AI-powered resume analysis, pipeline-based applicant tracking, real-time Socket.IO chat, and a smart career role suggestion engine into a single, cohesive platform accessible to all stakeholders.

14. Methodology / Technical Approach (check all that apply and describe below):

☐ Software/System Development ☐ Data Analytics/Mining

☐ AI/ML/DL Model Development

☐ HCI/Usability Testing ☐ Survey/Interviews ☐ Secondary Data Analysis

☐ Network Measurement ☐ IoT/Robotics ☐ Web Scraping/Crawling

☐ Security/Penetration Testing (with written authorization only)

Brief description (design, datasets, tools, procedures, evaluation):

I worked on developing JobLink, a complete full-stack intelligent recruitment platform, starting with building responsive and role-specific frontend interfaces for Job Seekers, Recruiters, and Administrators using React.js and Vite. From there, I implemented backend logic using Node.js and Express.js, designed the PostgreSQL database schema with custom model classes, and managed data flow across authentication, job management, application tracking, and

notification systems. I integrated a locally-hosted Python FastAPI ML Engine powered by Sentence-BERT to automate resume screening and generate hybrid ATS scores combining semantic similarity, skill overlap, and domain-specific boosting using the ESCO and O*NET datasets. I further built a real-time bidirectional chat system using Socket.IO with message persistence, read receipts, and avatar support, and implemented a Smart Career Match feature that recommends top job roles from any uploaded resume. By integrating all these components across four independent service tiers and testing the functionality thoroughly end to end, I was able to transform the concept into a fully working recruitment solution that users across all three roles could interact with seamlessly. This experience helped me understand how each layer of an application — from neural network inference and REST API design to real-time WebSocket communication and frontend state management — contributes to overall system performance and user experience.

_____

15. Software/Hardware & Infrastructure

- Primary languages/frameworks: React.js , next.js, node.js
- External libraries/models (names & licenses): _____
- Compute/Cloud services (provider, region, data location): _____
- On-prem resources/labs: _____

C. Data, Participants, and Ethics

16. Does the study involve human participants (e.g., usability tests, surveys, interviews, behavioral logging)?

☐ Yes ☐ No

If Yes, answer Q17–Q24. If No, skip to Q25.

17. Participant Recruitment (source, criteria, messages, incentives/honoraria):

_____

18. Anticipated Impact/Risks to Participants (time burden, privacy, discomfort, potential risks):

_____**19. Confidentiality & Anonymity (minimization, pseudonymization, access controls):**

_____**20.**

Informed Consent Procedure (consent form/script, withdrawal, parental consent, accessibility):

_____

21. Sensitive Contexts (tick if applicable):

☐ Personal or sensitive topics ☐ Children or vulnerable adults

☐ Possible psychological/social/physical harm or distress

☐ Audio/video recording or screen capture ☐ Geo-location tracking or telemetry

If ticked, justify risk-mitigation measures and provide protocols:

22. Data Collection Details (Human Studies)

- Instruments (surveys/interview guides/UX tasks): _____
- Duration per participant: _____ minutes
- N (target): _____
- Compensation (if any): _____

23. Debriefing & Support (post-study debrief, support contacts, complaint mechanism):

24. Privacy Notices (data use, storage, sharing, and participant rights):

If No Human Participants: Complete the Following

25. Datasets and Digital Sources

- Primary datasets/sources (name, URL/DOI, owner): _____
- Licenses/Terms of Use (attach evidence of permissible use): _____
- Provenance & Documentation (dataset cards/datasheets, model cards): _____

26. Web Scraping / Crawling (if applicable)

- Target domains/URLs: _____
- Respect for robots.txt and rate-limits: ☐ Yes ☐ N/A
- Terms of Service compliance & permission letters (attach): ☐ Attached
- Data minimization & exclusion of personal data: ☐ Ensured
- Anti-disruption safeguards (throttling, off-peak scheduling): ☐ Implemented

27. Security/Penetration Testing or Network Experiments (if applicable)

- Scope and assets under explicit written authorization (attach proof):
- ☐ Attached
- Non-interference & logging plan; rollback procedures: _____
- Legal and institutional policy compliance statement: _____

D. Data Protection & Management (All Projects)

28. Personal Data Handling (tick all applicable):

☐ No personal data processed ☐ Pseudonymized data only

☐ Personal data (contact info, identifiers) ☐ Special categories/sensitive data

29. Data Protection Compliance

- Applicable laws/policies (institutional policy, national data protection law, platform TOS): _____
- Legal basis/permission for processing (consent, license, contractual permission, legitimate use): _____

30. Data Lifecycle Plan

- Storage location (drive/server/cloud & region): _____
- Security (encryption, access control, keys/passwords): _____
- Retention period: _____
- Destruction method & date: _____
- Sharing (who will access; will de-identified data/code be released?):
- ☐ Yes ☐ No
- if Yes, repository/license and de-identification approach: _____

31. Bias, Fairness, and Safety (for AI/ML/NLP/CV)

- Expected risks (bias, representational harms, unsafe outputs): _____
- Mitigations (sampling, audits, human oversight, content filters, red-teaming): __
- Model reporting (model card/limitations/appropriate use): _____

32. Intellectual Property & Licensing

- Ownership of code/data/models: _____
- Open-source release plan and license (e.g., MIT, Apache-2.0, CC-BY): _____
- Third-party license compliance confirmed: ☐ Yes

33. Environmental & Cost Considerations (optional)

- Estimated compute (GPU hours/energy) and minimization steps: _____
- Budget/credits and cost controls: _____

E. Training, Conflicts, and Safety

34. Research Ethics/Compliance Training Completed?

☐ Yes (course/date): _____ ☐ No (commitment date: _____)

35. Conflicts of Interest (financial, employment, personal, or role-based)

☐ None ☐ Disclosed below: _____

36. Health & Safety (labs/IoT/robotics/fieldwork): risk assessment; emergency procedures and supervision

☐ Yes (attach) ☐ Not applicable

F. Declarations

Student Declaration

I, M Akash affirm that the information provided is accurate and complete. I understand that approval is required before commencing any data collection, scraping, security testing, or human-participant activities; that unauthorized testing or data acquisition is prohibited; and that any substantive change in design, methods, data sources, or risk profile requires re-submission for approval.

Signature (Student): _____ Date: _____

Supervisor (Faculty Guide) Approval

I, _____, have reviewed the proposal and ethics safeguards with the student. I verify that the project's scope, data handling, and risk mitigations are appropriate for MCA standards and institutional policies.

Signature (Supervisor): _____ Date: _____

DRAC/Department Member Review (if applicable)

Name: _____ Signature: _____

Date: _____

Industry/External Mentor Acknowledgement (if applicable)

Name & Organization: _____ Signature: _____

Date: _____

G. Submission & Processing

Please submit the signed form with all attachments (study instruments, consent forms, permissions, dataset licenses/TOS evidence, risk assessments, authorization letters for testing/scraping) to the Department Ethics Committee—Computer Applications (MCA/BCA).

Processing time: Please allow at least four weeks for review.

H. Office Use Only — Department Ethics Committee (MCA/BCA)

- Chair/Reviewer: _____
- Date Received: _____
- Review Date: _____
- Decision: ☐ Approved ☐ Minor Revisions ☐ Major Revisions ☐ Not Approved
- Conditions/Comments(required for any non-"Approved" decision):_____
- Follow-Up Required By (date): _____
- Final Sign-off (Chair): _____
- Date: _____

Attachments Checklist (tick if attached)

☐ Project synopsis/abstract

☐ Detailed methodology / system design

☐ Consent forms & participant materials (if any)

☐ Recruitment messages (if any)

☐ Dataset documentation & licenses / TOS permissions

☐ Web-scraping plan & permissions (if applicable)

☐ Security testing authorization (if applicable)

☐ Data management & retention plan

☐ Risk assessment (lab/IoT/robotics/network)

☐ Ethics/compliance training proof

Notes

- No unauthorized penetration testing, credential harvesting, denial-of-service, or intrusive scanning is permitted.
- Respect platform terms, robots.txt, and rate limits for any automated data collection.
- De-identify and minimize data, especially when any personal data may be incidentally captured.
- Document limitations and appropriate use for any released code/models/datasets.